

Where the Seconds Go:

Bounding Mixture-of-Experts Decode from Host Memory, and Measuring What Foresight Is Worth

Harshith Kantamneni
Independent Researcher

Abstract

A Mixture-of-Experts model too large for one consumer GPU keeps most of its experts in host memory, and decoding one token at a time then waits on host-memory reads. We bound the time per token of any system that runs the model’s exact routing with C GPU slots per layer: the fewest host reads any policy can make (Belady’s MIN with bypass) over the machine’s measured read rate. Our llama.cpp expert cache, faster than FreeToken at 11 of 12 configurations, reaches 25–43% of this bound, while a microbenchmark that replays MIN’s reads layer by layer, each free to take the CPU or the PCIe link, reaches at least 86% of the bound’s read time. To see where the rest of the time goes, we build oracles of the future routing into the engine and run them on rented RTX 5090 hosts, each host a sample. Knowing the future helps most when the cache spends it as MIN does: it caches only the experts MIN would cache, and loads each from host memory once, by a copy over PCIe, rather than running it on the CPU and copying it afterwards. Together the two changes close up to 60% of the gap to the bound, against at most 26% for either alone. On the one machine whose link is much slower than its CPU, the copy in the step loses and only the copy made ahead of use breaks even. A model of the two read paths, given a host’s bandwidth probe and the bytes each path carries, predicts the time of the deployed cache and of every state that copies in the step to a median of 2.2%. Finally, we degrade the oracle to price foresight: at the budgets where host memory binds, half of its saving in host reads needs exact routing of the next 2 to 10 tokens; across nine models, the horizon needed holds about $0.65C$ distinct experts per layer, which no online policy, learned admission order, speculative batch or forecaster we tested approaches.

1 Introduction

Open-weight Mixture-of-Experts (MoE) models now carry 30–120B parameters and activate a few billion per token; a desktop GPU has 24–32 GB. Running such a model for one user therefore means keeping most experts in host DRAM. Each decoded token reads its routed experts from GPU memory if they are resident, and otherwise from host memory: either the CPU runs the expert, or the

expert is copied over PCIe into a GPU slot [24, 20]. Many systems build on this: GPU expert caches with demand fetch [14, 52, 44], CPU execution of misses [30, 11, 64, 22, 63, 49], a per-miss choice between the two [55, 66], and expert prediction [23, 65, 50]. Each reports a speed-up over a baseline of its choosing. None states how fast the same machine *could* decode the model within the same GPU memory, or what the rest of the time is made of.

We answer both questions for batch-1 decode of gpt-oss-120b [40] and Qwen3-30B-A3B [54] on rented RTX 5090 hosts, with three findings.

(1) Where host memory binds, time is host bytes. The fewest host reads any exact-routing policy can make, divided by the machine’s measured host read rate, bounds the time per token (Section 3). Our cache reaches 25–43% of this bound; by one common procedure, published systems in the same class reach a median of 13.6% and ours 27%. The bound’s read time is nearly reachable: MIN’s reads, replayed layer by layer without the model and each free to take either path, reach 86–96% of it on three hosts. Requiring each expert to be read once, so that every admission crosses the link, raises the bound by at most 7%.

(2) Foresight pays most when spent as the optimum spends it, on a read path with room (Section 4). With the future routing known, caching only what MIN caches and copying each cached expert once, in the step, is 16–51% faster than our deployed cache on three hosts at six budgets, and 25–81% faster with the copies made a few steps ahead. Either change alone closes at most 26% of the gap to the bound; together they close up to 60%. The copy in the step needs a link that is not much slower than the CPU: on the one machine whose link reads at 0.29 of its CPU rate it loses, while the copy made ahead breaks even. A model of the two read paths, with one constant per budget fitted on other hosts, predicts the time of the deployed cache and every state that copies in the step within a median 2.2% on 14 hosts, and from the bandwidth probe alone within a median 3.8% on later launches. The usual ways of spending foresight, Belady prefetch and copying what the CPU has just served, read $1.6\text{--}2.5\times$ MIN’s bytes and at the smallest budgets gain at most 15%, or lose.

(3) Foresight has a measurable price (Section 5). We degrade the oracle to a window of the next W tokens,

exact or with each foreseen expert right only half the time. Added to a policy that admits every miss, 4 exact tokens recover 0.32–0.47 of MIN’s gain over that policy at the smallest gpt-oss budget on a 10-host panel, and 8 half-right tokens 0.20–0.26; measured against the deployed cache, such short windows often lose. Added to the deployed policy instead, 16 tokens break even or gain, at most 12%. On nine models’ traces, the window that closes half of the read gap holds about $0.65C$ distinct experts per layer, a one-constant rule that predicts a held-out model as well as a two-constant power law. No online policy, learned admission order, speculative batch or forecaster trained on routing histories comes near that horizon.

Contributions.

- A **bound on time per token** for exact-routing systems with C slots per layer, from MIN’s host reads and the machine’s probed rates, with its sensitivity and a microbenchmark of its read time, beside our cache, FreeToken, llama.cpp and 52 published measurements (Section 3).
- An **in-engine oracle factorial** that crosses what to cache with how each cached expert is loaded, on two hosts at six budgets and 13 more at two, with an order-free accounting of the gap, a test of launch-to-launch variation, and a model of the two read paths tested on hosts it was not fitted on (Section 4).
- A **price on foresight**: its value by horizon and accuracy, measured in the engine and on nine models’ traces, and a one-constant rule for the horizon in distinct experts (Section 5).
- An artifact in which every prediction was committed before the machine it concerns started, and every number in this paper is generated by a script (Appendices B and K).

2 Setting and Method

Models and workload. gpt-oss-120b (36 MoE layers of 128 experts, top-4, MXFP4, 13.25 MB per expert) and Qwen3-30B-A3B in BF16 (48 layers of 128 experts, top-8, 9.44 MB per expert). Prompts are the 30 AIME-25 problems, and the cache carries over between problems as in one user’s session. The comparison with other systems (Table 2) decodes 256 tokens per problem greedily through each system’s server, timed by one client; the oracle jobs replay each model’s own greedy output teacher-forced in process, so that every configuration sees the same routing. A *budget* is the share of each layer’s experts that fit in GPU slots (C of E); we use 11%, 25% and 40% for gpt-oss and 12.5%, 25% and 43.75% for Qwen3.

The engine. Our cache is a 3,500-line patch to llama.cpp [17]. Each MoE layer has C GPU slots. Routing is decided on the GPU; resident experts run there. A

Table 1: Cache configurations. *Reads*: how many times an admitted expert is read from host memory. All run in the same engine; the oracles read the future routing from a trace of the same text recorded on the same machine.

Name	What it admits, and how	Reads
<i>No foresight</i>		
deployed	decayed request count beats the lowest resident’s by $\kappa=1$; the CPU serves the miss, then a background copy fills the slot	2
single read	the same admissions, fetched into the slot in the step	1
admit every miss	every miss fetched into the lowest-scored slot (demand fetch)	1
<i>Oracles</i>		
MIN, 2 reads	MIN with bypass; the CPU serves the miss, then a copy	2
MIN, 1 read	MIN with bypass; admissions fetched in the step	1
MIN, prefetched	MIN, 1 read, plus copies issued three steps ahead	1
Belady	every expert of the coming steps, into the slot needed furthest ahead (also with 1 read, and prefetched)	2 or 1
window W	admit every miss unless the next W tokens show the victim needed sooner; victim the lowest-scored resident not seen in the window, else the one seen furthest	1
window W , recall r	the same, each future expert kept with probability r , else replaced by a random one	1
deployed + window W	the deployed policy with the same window and victim choice; the CPU serves the miss, then a background copy	2

non-resident expert, a *miss*, is read from host memory in one of two ways: CPU helper threads run it (the GPU signals them through mapped memory), or it is *fetched*, copied into a slot and run on the GPU in the same step. The deployed policy admits a miss into a slot when its decayed request count (half-life 16 tokens) beats the lowest resident’s by a margin $\kappa=1$. An admitted miss is first served by the CPU and then copied into its slot in the background, so it is read from host memory twice. How many of a layer’s misses to fetch rather than run on the CPU is a per-machine table, computed from the machine’s bandwidth probe by a calibrated time model (Appendix C).

Configurations. Table 1 names the policies we compare. The oracles read the routing of the rest of the problem from a trace recorded on the same machine over the same teacher-forced text; they decide, they do not predict. *MIN* is Belady’s optimum with bypass [5, 61]: a miss is admitted only if some resident’s next use comes later than the miss’s own, and that resident is evicted. With bypass allowed, maximising hits under C slots is an interval-colouring problem that this greedy rule solves exactly [10, 25].

Hosts and statistics. RTX 5090 cards (32 GB, 1.79 TB/s datasheet) in desktop hosts, rented by the hour. Every job first probes how fast the host reads host memory: from CPU threads (B_c), over PCIe (B_p), from both at once (B_{cp}), and the highest rate by any method (B_{host}). Time per token is averaged over problems, and ratios are ratios of those means, paired by problem (jobs 093–097 averaged speeds instead; no ratio moves by more than 0.006); within-host intervals are 95% percentiles of 10,000 bootstrap resamples of problems. Machines vary far more than problems or launches: two panel hosts with the same CPU model differ by 16–29% in the deployed cache’s time per token, while 3 panel machines relaunched later came within 1.9% of their first launch. A relaunch can still move a ratio by up to 0.030, more than a typical within-launch interval (median half-width 0.009), so on one host we do not read differences that small. The host panel of Sections 4 and 5 (the first 20 problems, one launch per host, configurations in a seeded random order with a cold cache each) therefore treats the host as the unit: pooled intervals come from 2,000 resamples of a crossed bootstrap that draws hosts and, for all of them at once, problems [29], and describe this sample of rentals. Each job’s predictions are in its script’s header, committed before the machine started; Appendix B scores the 565 clauses of jobs 073–098 by hand and the 273, 216, 25 and 33 of jobs 099–102 by machine.

Terms. A *budget* is the share C/E of a layer’s experts that fit in GPU slots; it is *host-bound* when host reads, not the GPU, bound the time. A *miss* is a requested expert not resident. An *admission* puts a missed expert into a slot, evicting a resident; MIN may also *bypass*, serving the miss without admitting it. An admitted expert is *loaded* in one of two ways: *copied in the step*, over PCIe before the GPU runs it (one host read, on the critical path), or *served by the CPU and copied in the background* (two host reads, the second off the critical path). The two *read paths* are the CPU’s own reads of host memory and the PCIe link; their probed rates give a host’s *link-to-CPU ratio* B_p/B_c . The *gap* is the deployed cache’s time per token minus the bound’s.

3 A Bound on Time per Token

Consider any system that executes the model’s exact routing with at most C experts of each layer resident on the GPU. Per token each of the Lk routed experts is either served by a resident copy, which the GPU reads from its own memory, or read from host memory: by the CPU that runs it, or by the copy that brings it into a slot. Let R^* be MIN’s host reads per token, summed over layers, on the routing trace; no policy, even one that knows the future, reads fewer. If c experts per token run on the CPU, host memory supplies at least $\max(R^*, c)S$ bytes of experts of S bytes each, and the GPU reads the dense weights D

plus $(Lk - c)S$. With everything overlapped,

$$\bar{T} \geq \min_{0 \leq c \leq Lk} \max\left(\frac{D + (Lk - c)S}{B_{\text{gpu}}}, \frac{\max(R^*, c)S}{B_{\text{host}}}\right). \quad (1)$$

A budget is *host-bound* when the minimum is at $c=0$: the GPU could take every expert, and host reads bind. That holds at the two smallest budgets of each model. The bound covers exact routing, whole experts of unmodified bytes, at most C residents per layer and one token per forward pass; it drops all latencies and lets the two read paths share B_{host} freely.

Table 2 places three systems against it on two hosts. Our cache leads FreeToken [55] at 11 of 12 configurations and runs 2.0–4.0× stock llama.cpp at equal GPU expert memory. Tuned over five settings on a third host, FreeToken gains at most 1.04× and still trails at 5 of six cells (Appendix I). Against the bound, our cache stands at 25–43%; with the GPU term at the rate all-in-VRAM decode reaches at batch size 1, at 31–56%.

How tight is it? The GPU term is the loosest: at batch size 1 the all-in-VRAM runs reach only 52% (gpt-oss) and 61% (Qwen3) of the datasheet rate. At that rate the bound is unchanged at the smallest budget of each model, where the host term binds, and moves at the others: at gpt-oss 25%, and at Qwen3 25% on host B, the GPU term takes over. Table 16 (Appendix I) varies the other assumptions one at a time, among them the host rate (the median of the probe’s samples instead of the highest); with all of them together our cache stands at 33–57% of the bound on host B. Pooling the LC slots across layers lowers the reads by 4.5–18.6%, which FreeToken’s single cache could exploit; we score it against the pooled bound (18–40%).

Can the read time be reached? The bound treats a token’s host reads as one stream at the host’s best rate. An engine reads whole experts, layer by layer, and each layer waits for its own. A microbenchmark (job 102) replays MIN’s per-layer reads of gpt-oss at 11% and 25% on three hosts without the model: CPU threads read part of each layer’s experts from host memory while the copy engine copies the rest into GPU slots, split at the probe’s rates (Table 17). Waiting for every layer, it reaches 86–96% of the bound’s host term; waiting once per token, 93–97%. A model of the same schedules from the probe’s rates predicts each mode’s time within 0.93–1.14×; on the 21 probes of jobs 095–101 it gives the per-layer schedule 85–99% at 11%, though on the slow-link Intel host it was 12 points optimistic (98% against 86% measured). The microbenchmark’s CPU threads only read; the engine’s helpers also compute, at 1.09–1.31× the probe’s time per expert (Appendix C).

The free split ignores one constraint. Read once, an admitted expert must cross the link to reach its slot. MIN has many hit-optimal schedules, and they differ in how many

Table 2: Decode speed at equal GPU expert memory on two RTX 5090 hosts (30 AIME-25 problems, the first 256 greedy tokens, one session). Ratios are of mean speeds, paired by problem, with 95% bootstrap intervals. Our cache uses each machine’s fetch table; FreeToken uses its faster backend per budget, picked on host B on a separate launch. *Bound*: Eq. (1) on that machine (MIN’s reads, the host’s highest probed rate) with the GPU at its datasheet rate, and at the rate all-in-VRAM decode measures at batch size 1 (52% of datasheet for gpt-oss, 61% for Qwen3; Table 16 varies the other assumptions). Against the pooled bound their designs allow, FreeToken (one cache for all layers) and llama.cpp (whole layers pinned) stand at 18–40% and 8–20%. With every weight in GPU memory (an RTX PRO 6000) stock llama.cpp decodes gpt-oss at 255 and Qwen3 at 178 tok/s. [†]Selection launch only, which favours FreeToken.

Model	Experts on GPU	llama.cpp (tok/s)	FreeToken (tok/s)	Ours (tok/s)	Ours ÷ FreeToken (95% CI)	Ours ÷ llama.cpp	Bound, datasheet tok/s	GPU ours	Bound, measured tok/s	GPU ours
Host B: Ryzen 9 9950X3D, card at 17001 MHz memory clock, CPU 72 GB/s, link 53, together 78, highest (the bound’s rate) 88										
gpt-oss-120b	11%	34.9	54.0 (hybrid)	69.9	1.294 [1.278, 1.312]	2.00×	172	41%	172	41%
gpt-oss-120b	25%	40.0	85.6 (hybrid)	109.2	1.275 [1.254, 1.295]	2.73×	430	25%	279	39%
gpt-oss-120b	40%	47.7	132.2 (offload)	152.5	1.154 [1.134, 1.172]	3.20×	519	29%	279	55%
Qwen3-30B-A3B	12.5%	19.5	38.7 (hybrid)	40.0	1.032 [1.022, 1.042]	2.05×	96	42%	96	42%
Qwen3-30B-A3B	25%	22.1	54.7 (hybrid)	63.1	1.153 [1.134, 1.173] [†]	2.86×	214	30%	192	33%
Qwen3-30B-A3B	43.75%	28.4	103.1 (offload)	108.2	1.049 [1.036, 1.064]	3.81×	308	35%	192	56%
Host S: Ryzen 9 9950X at the stock clock, card at 14001 MHz, CPU 56 GB/s, link 53, together 66, highest 71; FreeToken’s backend carried over from host B										
gpt-oss-120b	11%	28.9	48.0 (hybrid)	57.9	1.207 [1.195, 1.219]	2.00×	140	41%	140	41%
gpt-oss-120b	25%	33.6	78.7 (hybrid)	94.1	1.196 [1.163, 1.224]	2.80×	351	27%	275	34%
gpt-oss-120b	40%	40.1	122.2 (offload)	133.6	1.093 [1.077, 1.111]	3.34×	514	26%	275	49%
Qwen3-30B-A3B	12.5%	16.1	32.7 (hybrid)	33.5	1.027 [1.018, 1.036]	2.08×	78	43%	78	43%
Qwen3-30B-A3B	25%	18.5	51.4 (hybrid)	53.9	1.048 [1.037, 1.059]	2.91×	174	31%	174	31%
Qwen3-30B-A3B	43.75%	23.8	98.3 (offload)	95.8	0.974 [0.962, 0.987]	4.02×	305	31%	189	51%

experts they admit: the greedy rule of Section 2, which our oracles follow, admits 21.8 experts per token at 11% (57% of its reads), while the fewest any hit-optimal schedule needs is 12.4 (32%; a linear program over reuse intervals whose solutions are integral, `scripts/linkaware.py`). A bound that sends every admission over the link at the probe’s highest link rate, and trades misses against admissions freely, is at most 1% above the bound’s host term at 11% and 7% at 25% on the 24 probes of jobs 095–102, both largest on the machine whose link reads at 0.29 of its CPU rate. Reading each expert once therefore barely loosens the bound; which hit-optimal schedule an engine copies does matter (Sections 4 and 7).

Published systems. We scored 52 published batch-1 measurements from 13 sources with one procedure: the same bound at each paper’s datasheet ceilings, MIN’s reads from our traces of its model, every efficiency 1 (`scripts/audit.py`; every row in Appendix J). The 20 rows inside the bound’s class reach a median 13.6% of it (quartiles 8.1–20.6%). Scored by the same procedure, our cache reaches 18–42% (median 27%) on the two hosts of Table 2. Either way, systems in this class run several times slower than the routing allows on the same machine; the next section asks what the missing time is made of.

4 Where the Seconds Go

The deployed cache does not know which experts the coming tokens will need. We measure what knowing them is worth by running oracles inside the engine: on three hosts (O3–O5) at all six budgets, on a panel of 10 hosts at the two host-bound gpt-oss budgets, and on follow-up

launches. Figure 1 shows the speed of each state against the deployed cache on the same host.

Spent as the optimum spends it, foresight pays.

MIN with each admission copied once, in the step, is faster than the deployed cache at every budget on hosts O3–O5, by 16–51%. Making the same copies a few steps ahead lets them overlap the steps before, for 25–81% in all. Even this best oracle reaches only 46–73% of the bound at the host-bound budgets on O4 and O5; the rest of this section asks why.

Spent the usual ways, it pays less. Two common ways of using foresight waste host reads: a Belady prefetch also admits experts MIN would bypass, and serving a miss on the CPU and then copying it reads every admission twice. They read 1.6–2.5× MIN’s bytes and, at the smallest budget of each model on O4 and O5, gain at most 15% over the deployed cache or lose, where MIN with one read gains 1.16–1.44×. The trade-off between prefetching what an optimum would bypass and admitting conservatively is classical for disks [8] and CPU caches [26].

The machine decides which load pays. A copy in the step puts the read on the PCIe link and on the critical path, so it pays only where the link has room. Figure 2 plots each state’s gain against the host’s link-to-CPU read ratio. MIN with one read gains less as the ratio falls (correlation 0.93 with its log at 11%), and on the one machine whose link reads at 0.29 of its CPU rate it loses, on both of its launches. Made ahead of use, the same copies do not lose anywhere; on that machine they break even. Admitting every miss, which sends every read



Figure 1: What foresight is worth in the engine, by how it is spent: speed of each configuration of Table 1 relative to the deployed cache on the same host, at the six budgets ([†]host-bound). Hosts O3, O4 and O5 at every budget (O4 and O5 in a randomised order); the 10 panel hosts (grey ticks) at the two gpt-oss host-bound budgets, and 3 of them at Qwen3 12.5%. The prefetched Belady variant and the learned admission order are in Appendices F and G. Within-host 95% intervals are mostly narrower than the markers. On these hosts admitting what MIN admits and reading it once (*MIN, 1 read*) gains at every budget; the usual ways of spending foresight (*Belady, MIN, 2 reads*) gain at some budgets and lose at others. The panel machine whose link reads at 0.29 of its CPU rate is the exception (Fig. 2).

over the link, is at best at parity with the deployed cache (0.42–1.00 $\times$).

A model of the two read paths. Let X_c and X_p be the bytes per token that the CPU and the PCIe link read from host memory in a given state (from its counters), and B_c , B_p and B_{cp} the host’s probed rates for each path alone and for both at once. The calibrated model of Appendix C predicts $T = G + \max(X_c/B_c, X_p/B_p, (X_c+X_p)/B_{cp})$, where G , the GPU’s work and the step’s fixed costs, is one constant per budget, taken for each host as the median of the other hosts’ fits. Three tests:

- *Given each state’s byte counts*, it predicts the time of the deployed cache and of every state that copies in the step within a median 2.2% (90th percentile 10%) on 14 hosts.
- *From the probe alone*, written on the host before any timed run, it predicted the four in-step states whose byte counts do not depend on the host within a median 3.8% on later launches. Its two large errors have causes: copies in the step share host memory less evenly than the probe’s combined rate says, which costs up to 19% on Pd, whose link reads at 0.41 of its CPU rate (Appendix C), and on one new host the probe read the link more slowly than the engine’s copies ran (up to 20%).
- *Whether the copy in the step pays* it gets right on every host, but that test is weak: the copy loses on only one machine, so the rule “always copy” and a

model with one read path get every other host right too. What the second path buys is that machine’s losses, which it predicts on both launches.

The model does not resolve effects of a few percent (it puts the single read within 2% of the deployed cache; measured, 0.90–1.03 $\times$), and it predicts the oracle states whose copies run in the background or ahead too fast, by a median 16%: those copies are not free.

An order-free accounting. Two choices separate the deployed cache from MIN with one read in the step: *what to cache* (the deployed policy’s admissions or MIN’s) and *how to load* a cached expert (served by the CPU and then copied in the background, two reads; or copied in the step, one read). The engine ran all four combinations on O4, O5, the panel and three new machines. Take O4 at gpt-oss 11%, 10.6 ms per token from the bound. Changing only what to cache closes 4.5% of that gap; changing only how to load closes 3.9%; changing both closes 51.5% (rounded). The extra 43.2 points are an interaction: MIN caches 4.6 times as many experts per token as the deployed policy, and loaded the deployed way each is read twice, so their background copies crowd the reads of the steps that follow. Because each choice’s effect depends on the other, we credit each with its effect averaged over the two orders, its Shapley value [16, 27]: 26.1% and 25.5% here. Splitting the interaction equally is a convention, so Table 3 also lists each choice alone and the interaction. Copying ahead needs foresight by construction, so it is nested under

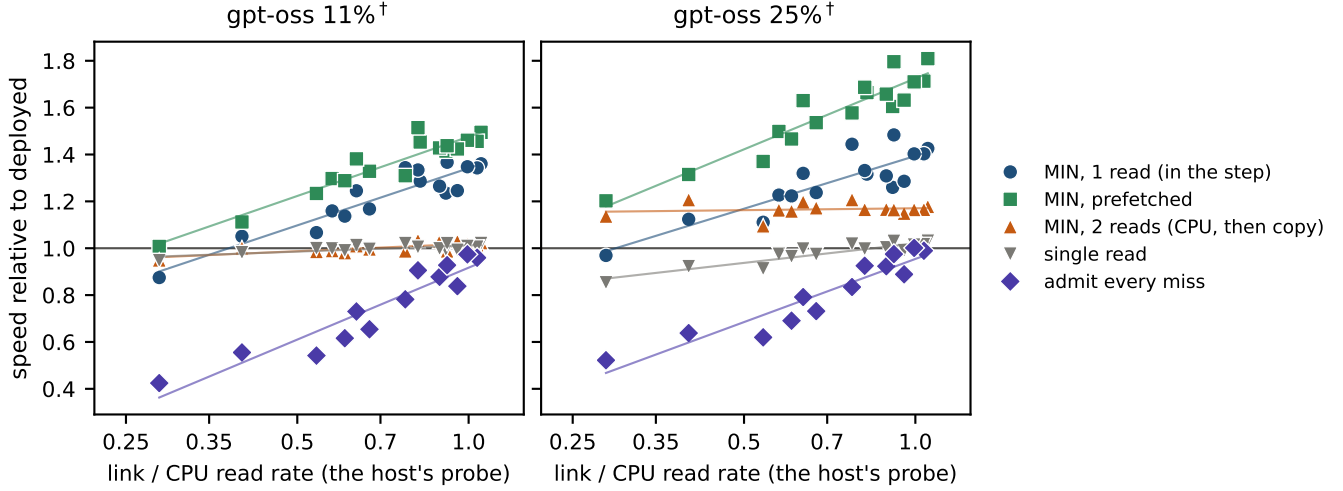


Figure 2: Which way of spending foresight pays depends on the machine: speed relative to the deployed cache on the same host against the host’s link-to-CPU read-rate ratio from its own probe, at the two gpt-oss host-bound budgets, on hosts O3–O6, the panel and the new machines of the follow-ups. Lines: least-squares fits in the log of the ratio, for the trend only.

Table 3: An order-free accounting of the gap between the deployed cache and the bound at the host-bound budgets, as percentages of the gap (ms per token). The engine ran both choices of what to cache (the deployed policy’s admissions or MIN’s) with both ways to load a cached expert (two reads: the CPU serves it, then a copy; one read: a copy in the step). *Load alone* and *cache alone* are each choice’s effect with the other at the deployed level; the *interaction* is how much more MIN’s set is worth loaded once than loaded twice. The Shapley values average each choice’s effect over the other’s two levels and add up, with the prefetch (nested under MIN’s set, since it needs foresight) and the rest, to 100%. Panel: mean over its hosts (number in parentheses) and, in small type, the range over its hosts (not an interval); a fast link reads at least half as fast as the host’s CPU, a slow one less. Last rows: the new machines of the follow-up launches, with their link-to-CPU ratio.

Host	Budget	Gap (ms)	Load alone	Cache alone	Interaction	Shapley value cache	load	Prefetch	Rest
O4	gpt-oss 11%	10.6	4	4	43	26	25	13	36
O4	gpt-oss 25%	8.5	4	22	17	31	13	22	34
O4	Qwen3 12.5%	18.4	3	−1	58	28	32	4	36
O4	Qwen3 25%	14.1	2	14	37	33	21	17	30
O5	gpt-oss 11%	11.4	0	−2	28	12	14	17	57
O5	gpt-oss 25%	9.1	−4	20	10	25	1	21	52
O5	Qwen3 12.5%	19.8	1	−13	43	9	23	7	62
O5	Qwen3 25%	15.3	0	14	17	22	9	16	53
Panel, fast link (8)	gpt-oss 11%	11.6	1 (−2 to 4)	3 (−2 to 6)	38 (30 to 47)	22 (14 to 26)	20 (14 to 27)	13 (−4 to 20)	45 (38 to 55)
Panel, fast link (8)	gpt-oss 25%	9.0	0 (−4 to 3)	22 (18 to 26)	15 (10 to 19)	29 (25 to 35)	8 (1 to 12)	20 (9 to 23)	43 (38 to 49)
Panel, slow link (2)	gpt-oss 11%	9.7	−6 (−9 to −3)	−4 (−9 to 0)	3 (−6 to 11)	−3 (−11 to 6)	−5 (−12 to 2)	17 (9 to 24)	91 (83 to 99)
Panel, slow link (2)	gpt-oss 25%	7.8	−17 (−22 to −11)	19 (16 to 23)	3 (3 to 3)	21 (17 to 25)	−15 (−21 to −10)	22 (17 to 26)	73 (68 to 78)
5700X3D (0.92)	gpt-oss 11%	14.9	4	−2	55	25	31	8	36
5700X3D (0.92)	gpt-oss 25%	11.5	4	22	24	34	17	18	31
9950X, x8 (0.54)	gpt-oss 11%	8.3	0	−3	17	5	8	28	59
9950X, x8 (0.54)	gpt-oss 25%	7.4	−15	13	17	22	−6	26	59
9800X3D, slower link (0.61)	gpt-oss 11%	10.8	−2	−4	29	11	12	20	57
9800X3D, slower link (0.61)	gpt-oss 25%	8.9	−5	20	12	26	1	20	54

MIN’s set (12.7% here); what no oracle touches is the rest (35.7%).

Across the 13 hosts whose link reads at least half as fast as their CPU, either change alone closes at most 26% of the gap, both together 14–60%, and the rest is 30–62%. On the 2 hosts below that line, loading in the step costs time at 3 of 4 host-budgets and the rest is 68–99%. We drew the line after the panel ran; the model above, not the line, carries the prediction.

The rest. What no oracle touches has three visible parts. The prefetch is imprecise: over all six budgets it reads 1.18–1.58× MIN’s bytes, from victims chosen steps early and from misses inside the lead. The overlap is partial: a model that charges every host read in sequence with the GPU’s work over-predicts the prefetching state’s time by +1 to +38%, so some reads overlap and some do not. And the GPU runs at about half of its datasheet rate at batch size 1, which is most of what remains at the

GPU-bound budgets (Appendix F). The rest grows as the link slows relative to the CPU: on a slow-link host every state either puts reads on the slow link or leaves them to the CPU helpers, which take $1.09\text{--}1.31\times$ the probe’s time per expert (Appendix C). On those hosts part of the rest may be the oracle’s own admissions: the greedy MIN it follows admits 21.8 experts per token at gpt-oss 11%, where 12.4 give the same hits (Section 3), and copied in the step each crosses the slow link; we have not run a minimum-admission oracle. Little of the rest is slack in the bound: splitting MIN’s reads between the two paths at their probed rates tightens it by only 1–9% of the gap on the fast-link hosts and 0–2% on the slow-link ones, and sending every admission over the link raises it by at most 7%.

Machines differ more than problems. On the panel, the within-host 95% interval of MIN-with-one-read’s gain has a median half-width of 0.009 at gpt-oss 11%, while its standard deviation across hosts is 0.154; the two-stage interval over hosts is [1.12, 1.30]. Part of that spread follows the link-to-CPU ratio (Fig. 2) and part does not, while a relaunch of a machine reproduces its ratios within 0.030 (Section 2). The machine, not the launch or the problem, is the unit of variation, so the statements of this section are about orderings that hold on every host of a stated class, and about a model that predicts each host.

5 How Much Foresight Is Needed

The oracles know the whole future; a forecaster would know part of it, imperfectly. We give the cache a *window*: the routing of the next W tokens, exact or with errors. Against the deployed cache, a window copied in the step pays only where the link has room: at 16 tokens it runs $0.71\text{--}1.31\times$ the deployed cache at gpt-oss 11% on the panel. Given to the deployed policy instead, a 16-token window breaks even or gains on every host, slow link included, but at most 12%; shorter or half-right windows run $0.94\text{--}1.03\times$. In host reads, half of MIN’s saving at the host-bound budgets needs exact routing of the next 2 to 10 tokens, a horizon no source of foresight we tested reaches.

Window policies. A window can be given to either way of loading of Section 4 (Table 1). *Window* extends admitting every miss and copies each admission in the step. At each step, for each miss in request order:

1. take a free slot if there is one;
2. otherwise pick the victim: the lowest-scored resident not requested in the next W tokens, or, if every resident is requested there, the one requested furthest ahead;
3. admit the miss unless the window shows the victim needed before the miss.

Table 4: Foresight on each read path (job 100): speed relative to the deployed cache on the same host. *In step*: admit every miss with a window of W exact tokens, each admission copied in the step (as on the panel). *Deployed path*: the deployed policy with the same window, its misses split between the CPU and an in-step copy by the host’s fetch table and its admissions copied in the background; $W=8$ at recall 0.5 in the column marked r . Hosts sorted by the probe’s link-to-CPU ratio.

Host	Link/ CPU	Budget	In step		Deployed path		r	MIN, 2 reads		MIN, 1 read	MIN, pref.
			$W=4$	16	4	16					
Pd again	0.41	gpt-oss 11%	0.66	0.89	1.02	1.03	0.94	1.00	1.05	1.11	
Pd again	0.41	gpt-oss 25%	0.69	0.81	1.03	1.12	1.01	1.19	1.11	1.29	
9950X, slower link	0.54	gpt-oss 11%	0.65	0.89	1.01	1.01	0.96	0.99	1.07	1.23	
9950X, slower link	0.54	gpt-oss 25%	0.69	0.80	0.96	1.06	1.00	1.10	1.11	1.37	
5700X3D	0.92	gpt-oss 11%	1.09	1.32	0.99	1.00	0.95	0.99	1.37	1.44	
5700X3D	0.92	gpt-oss 25%	1.06	1.23	1.03	1.07	1.00	1.16	1.48	1.80	
Ph again	0.96	gpt-oss 11%	0.99	1.20	1.01	1.03	0.98	1.03	1.27	1.45	
Ph again	0.96	gpt-oss 25%	0.97	1.09	1.03	1.07	1.01	1.16	1.29	1.64	

With no window this is admitting every miss, the online policy that reads least at these budgets (below); with the rest of the problem in view it is MIN with one read. *Deployed + window* gives the same window and victim choice to the deployed policy, so that foresight changes only what is cached: the host’s fetch table still splits the misses between the CPU and an in-step copy, and admissions are copied in the background. To degrade a window, each future expert is kept with probability r and otherwise replaced by a random one, so precision and recall are both r , with errors drawn independently at every step. The engine plans each step on the host before the step, exactly as a replay script does (equal on a CPU build at three budgets, within 1.7% on the panel’s GPUs).

Against the deployed cache. Table 4 runs both policies on 4 hosts (two panel machines relaunched and two new, link-to-CPU ratios 0.41–0.96). On the deployed path, a 16-token window runs $1.00\text{--}1.03\times$ the deployed cache at gpt-oss 11% and $1.06\text{--}1.12\times$ at 25%, about what MIN with two reads gains; shorter or half-right windows lose up to 6%. In the step, the same window beats the deployed path on the 2 hosts whose link reads at least 0.92 of their CPU rate and loses to it on the 2 whose link reads at most 0.54 of it. Short windows in the step often lose outright: on the 10-host panel a 4-token window is slower than the deployed cache on 6 hosts at 11% (8 at 25%), a 16-token window on 2 (3). The one spend of foresight that gains substantially wherever the link has room, and loses beyond its interval on no host we ran, is MIN with its copies made ahead (at worst $0.997\times$, on the lowest-ratio machine). On the deployed path the engine misses 7–33% more than a replay that admits at once, while the deployed state matches its replay within 2.4%; background copies that land late are the likely cause, which we have not isolated.

Against admitting every miss. The policy a window extends is the natural yardstick for what its foresight is worth, though a weak one in time. Figure 3 gives the

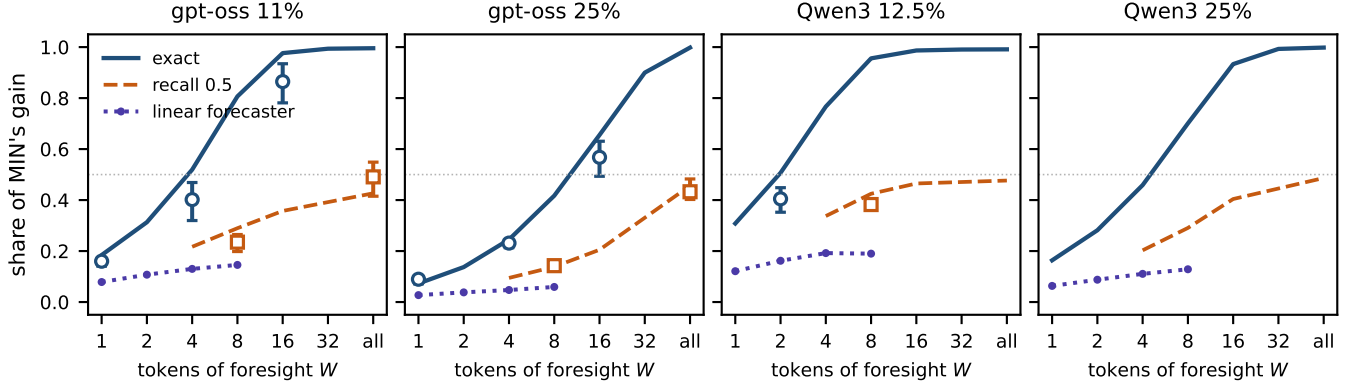


Figure 3: The value of partial foresight at the four host-bound budgets: the share of MIN’s gain over *admit every miss* recovered by exact routing of the next W tokens (solid), by a window whose experts are each right with probability 0.5 (dashed), and by a linear forecaster fitted on other text (dotted), in host reads on the AIME routing the engine runs; markers: the same share in time per token on the panel hosts (mean, and range over hosts; 10 hosts for gpt-oss, 3 for Qwen3 at 12.5%; no Qwen3 run at 25%). The grey line marks half of the gain.

share of MIN’s gain over admitting every miss that each window recovers, in time on the panel and in reads on the trace. At gpt-oss 11%, exact windows of 1, 4 and 16 tokens recover 0.14–0.17, 0.32–0.47 and 0.78–0.93 of the gain in time across hosts. At 25%, where the cache turns over more slowly, the shares are smaller: 0.21–0.24 at 4 tokens and 0.49–0.63 at 16. The calibrated model predicts these shares within a median 0.03 (at most 0.10, over 60 host-budget-windows).

Where the link is slow, time lags reads. A window saves host reads but, like the policy it extends, still copies almost every miss over PCIe in the step: at 11% a 4-token window removes 0.52 of the reads MIN removes, yet 97% of its remaining misses cross the link on the critical path, against 50% of MIN’s. Its share of the time gain is therefore below its share of the read saving, by 0.05 on hosts whose two paths are equally fast and by 0.20 on the host whose link reads at 0.29 of its CPU rate (correlation with the log ratio -0.91).

Accuracy matters as much as horizon. With each foreseen expert right only half the time, an 8-token window recovers 0.20–0.26 of the gain at 11%, in reads about what 2 exact tokens buy (on Qwen3 at 12.5%, where the engine ran both, 0.37–0.39 against 0.35–0.45). The whole future at half accuracy recovers 0.42–0.55. Against the deployed cache, the half-right 8-token window loses on 8 of 10 panel hosts at 11%, and on the deployed path on all 4 (0.94–0.98 $\times$). For the whole future at half accuracy the model over-predicts the share by up to 0.24 on fast-link hosts, which we have not explained; planning costs at most 277 μ s per step, too little to account for it. On the traces, dropping experts without replacing them (recall r , precision 1) costs about the same as replacing them for windows of up to 16 tokens, and less with the whole future in view at low recall (Appendix H).

On the traces of nine models. The trace study replays each model’s own sampled text (33,000–152,000 tokens per model) through an event-level simulator in which a token requests its k experts per layer as one set and no expert of the current token is evicted to serve it [62]. Adding *admit every miss* to the online policies changes their ranking: it reads least of LRU, LFU, two decayed-frequency variants, ARC, S3-FIFO and itself in 18 of 27 budget points (5 ties), and at most 2.7% more where it loses. Even so, the best online policy reads 35–110% more than MIN (Table 13, Appendix G). Against this stronger baseline, the window that closes half of the read gap grows with the cache’s turnover time C/k as $W_{50} \approx 0.67 (C/k)^{1.31}$ tokens ($r=0.977$ over 26 points; the exponent’s 95% interval over a bootstrap of models is 1.10–1.43, and 1.35 without the points below one token).

The horizon in distinct experts. A power law in tokens hides why the horizon differs between models. Paging theory measures lookahead in distinct future pages, not requests [2, 6]. Let $D(W)$ be the mean number of distinct experts a layer requests in a window of W tokens. At every point of the fit, $D(W_{50})$ is close to a fixed fraction of the cache: median 0.65 C (quartiles 0.61–0.72, coefficient of variation 0.16), while W_{50} itself ranges from 0.6 to 34 tokens. The models differ in how fast $D(W)$ grows ($D \propto W^\beta$ with β from 0.48 to 0.78), which is why the exponent in tokens wanders. Fitted on eight models and tested on the ninth, the rule “ $D(W_{50}) = cC$ ” predicts the held-out model’s horizon about as well as the power law, with one constant instead of two, and better than $W_{50} \propto C/k$ (Table 5); over a bootstrap of models the ratio of the power law’s median error to the rule’s is 0.95–1.09, so the two are not distinguished. Within a model the fraction falls as the cache grows (from the smallest budget to the largest in 9 of 9 models), so c is an average over

Table 5: Predicting a held-out model’s horizon W_{50} (fitted on the other eight models, tested on the ninth, over all 26 budget points): error as a factor, $\max(\hat{W}/W, W/\hat{W})$.

Rule	Constants	Median	90th pct.	Max
$W_{50} \propto C/k$	1	1.22	1.57	2.42
$W_{50} = a(C/k)^b$	2	1.19	1.51	1.67
$D(W_{50}) = cC$	1, and $D(W)$	1.16	1.42	1.61

budgets. What the rule adds is the unit: a forecaster’s target is stated in quantities it controls, the next $\approx 0.65C$ distinct experts of each layer.

What could supply it. We know of no realisable source of routing foresight that reaches this horizon. *Online policies* cannot (above). A *learned admission order* fitted on other text, with features the engine has when it plans a step, reads 6–13% fewer experts in the engine than the deployed policy with single reads, but runs $0.90\text{--}1.05\times$ the deployed cache at the host-bound budgets; on the trace, admitting every miss alone gets most of its read saving on Qwen3 (Appendix G). *Speculative batches* request a layer’s experts for K draft positions at once; once the rejected drafts’ experts are counted, they save an online cache 8–12% of its reads at 90% acceptance and cost 18–20% at 80% [12, 34, 41]. *Layer-ahead predictors* apply the next layer’s router to the current hidden state [23, 65]; they reach 84% recall at k and 97% with 8 candidates on gpt-oss in our engine, but see less than one token ahead. *Forecasters* trained on routing histories are the remaining candidate. A linear forecaster we fitted on each model’s other text forecasts gpt-oss’s experts with a precision of 0.48 one token ahead and 0.26 four ahead, and closes 3–15% of the read gap (6–19% on Qwen3; Fig. 3, dotted). Its precision falls with the horizon, and over a window it closes no more than a window with random errors of the same average precision; on Qwen3 less, because the experts it gets right are mostly ones the cache already holds. A sequence model reports 90% recall one token ahead on both of our models with $k+3$ candidates [50]; a recurrent model of the same inputs that we trained the same way as the linear one reaches 0.53 on gpt-oss and 0.54 on Qwen3 by that measure, and closes no more of the gap (Appendix H).

6 Related Work

Offloading systems. Demand-paging systems cache experts per layer and copy misses over PCIe [14, 52, 44, 46, 42]; an upstream llama.cpp pull request adds such a cache, admitting every miss into an LRU [18]. CPU-execution systems run non-resident experts on the host [30, 11, 64, 63, 49, 1]; a set-associative GPU cache with CPU-executed misses [22], a llama.cpp fork [32, 33] and two llama.cpp pull requests [39, 13] are the closest designs to ours. FreeToken [55] and DALI [66] split each

layer’s misses between CPU and PCIe. Predictors guess upcoming experts from the previous layer’s state [23, 65], several tokens ahead with a learned forecaster [50, 53], or from draft tokens [12, 34, 41]; Read-ME decouples the router so routing is known in advance [7]. Batched-throughput systems [43, 9, 56] and SSD tiers [36, 38] are outside our scope.

Bounds, oracles and caching theory. WiSP [59] bounds decode by PCIe turnover, Budgeting Bytes [58] by bytes over bandwidth for a given fetch set, and MoE-CAP [28] measures bandwidth utilisation against a single peak; the roofline [51] is the single-memory ancestor of our bound. Zhang [62] proposes a trace-level evaluation contract and reports a 44–46% gap between causal policies and MIN with bypass; Liang et al. [37] measure, across 20 models, the hit rate of an oracle cache that sees m tokens ahead against the cache ratio C/k . We add the time domain: an oracle inside a real engine, the cost of how foresight is spent, and a horizon rule that predicts held-out models. SAEM [57] gives its own stage-level planner perfect knowledge of the coming stage in its engine and finds that worth at most about 14% on an A100; SpecMD [19] benchmarks expert-cache and speculative prefetch policies across hardware; Angelopoulos et al. [3] give competitive bounds for layered expert paging. With bypass, MIN maximises hits exactly [10, 25]; when bypass and admission cost differ, the time-optimal placement is a different problem [60], which is why we use MIN’s reads as an input to a bound rather than as a time-optimal policy. Lookahead in paging is valuable when measured in distinct pages [2, 6], and learning-augmented paging is most robust with predictions of whether a page will be evicted before its next use [4]; learned replacement imitates Belady in CPU and content-delivery caches [25, 45] and in an SSD-backed expert cache [31].

7 Limitations

- **Hardware.** Rented consumer cards (RTX 5090 for the headline, a 4090 and a 3090 in Appendix D) in desktop hosts; GPU clocks could not be locked and power limits differed. The panel covers 10 hosts of several CPU classes, one launch each (3 relaunched later), at two gpt-oss budgets (Qwen3 at one budget on three); the crossed 2×2 ran at every budget on two hosts. Only 2 panel hosts have a link slower than half their CPU rate, and the copy in the step loses on only one machine (twice, on two launches), so the crossover in Fig. 2 rests on few points; the calibrated model, not the fit, carries the prediction there. That line at half was drawn after the panel ran; the panel’s pre-registered classes were by absolute link rate.
- **Workload.** Batch-1 decode of two models on AIME prompts, which were also used while developing the system; teacher-forced on each model’s own greedy

text; the panel ran the first 20. Prompts and batches above one bypass the cache.

- **The bound.** Exact routing and per-layer slots only; pooling slots across layers lowers MIN’s reads by 4.5–18.6%. It drops latencies and lets the two read paths share the host’s best probed rate freely, so it is optimistic, and a faster reader than our probe could exceed it. The best oracle comes to 46–73% of it. Its read time is nearly reachable in a microbenchmark that runs no model and whose CPU threads only read, on three hosts; whether the GPU’s work overlaps the reads as the bound assumes we have not measured, sending every admission over the link raises it by at most 7%.
- **The time model.** Its one constant per budget is fitted on other hosts; with the constants of the original blind test frozen, it over-predicts Qwen3’s speed on later hosts by +5 to +21% (Appendix C). It takes background copies as free, which they are not, and it fails on a 12-channel server host, where a per-layer latency floor, not memory, binds.
- **The oracles and the degraded windows.** Oracles read a recorded trace, and follow MIN’s greedy rule, which admits 21.8 experts per token at gpt-oss 11% where a hit-optimal schedule needs only 12.4. “MIN’s set” in the 2×2 is therefore one hit-optimal set among many; a minimum-admission set would put fewer copies on the link in the step and read fewer experts twice, which may change the slow-link results and the size of the interaction. Degraded windows draw errors uniformly and independently; a real forecaster’s errors are correlated with what the cache already knows, so the value of a forecaster at a given precision and recall can be lower than Fig. 3 shows. The windows of the panel extend admitting every miss and copy each admission in the step, which is the wrong path on slow-link hosts; on the deployed path the same windows gain little anywhere (job 100, four hosts).
- **Outputs.** Parity is measured by teacher-forced KL divergence and agreement against an all-VRAM reference (mean KL 0.0019 and 0.0005 nats), not by task accuracy.

8 Conclusion

With experts in host memory, batch-1 MoE decode is bound by host bytes. MIN’s reads, replayed alone with each free to take either path, reach at least 86% of the bound’s read time, and reading each expert once moves the bound by at most 7%, yet our cache reaches 25–43% of the bound, and by one common procedure published systems reach a median of 13.6% and ours 27%. Knowing the future recovers part of the gap when it is spent as MIN spends it, caching only what MIN caches and loading each expert once on a path with room: on hosts whose link

is not much slower than their CPU, 14–60% of the gap. Which path has room is a property of the machine that its two read rates predict; where the link is slow, copying the oracle’s admissions in the step overloads it. The rest of the gap on the other hosts, 30–62%, includes an imprecise prefetch, partial overlap and a GPU at half its datasheet rate. And foresight has a measurable price: about 0.65 C distinct experts of each layer’s coming routing, a few to tens of tokens ahead, at accuracies no forecaster we know of or trained reaches. That is the target for the next MoE cache, and for router designs that would make routing predictable in advance.

References

- [1] ATSIInfer: Automated tensor scheduling for hybrid CPU-GPU LLM inference on consumer devices, 2026. arXiv:2607.10183.
- [2] Susanne Albers. On the influence of lookahead in competitive paging algorithms. *Algorithmica*, 18(3):283–305, 1997.
- [3] Spyros Angelopoulos, Loris Marchal, Adrien Obrecht, and Bertrand Simon. Cache management for mixture-of-experts LLMs. In *Euro-Par*, 2025. arXiv:2509.02408.
- [4] Antonios Antoniadis, Joan Boyar, Marek Eliáš, Lene M. Favrholdt, Ruben Hoeksma, Kim S. Larsen, Adam Polak, and Bertrand Simon. Paging with succinct predictions. In *International Conference on Machine Learning (ICML)*, 2023.
- [5] Laszlo A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2):78–101, 1966.
- [6] Dany Breslauer. On competitive on-line paging with lookahead. Technical report, BRICS, 1995.
- [7] Ruisi Cai, Yeonju Ro, Geon-Woo Kim, Peihao Wang, Babak Ehteshami Bejnordi, Aditya Akella, and Zhangyang Wang. Read-ME: Refactorizing LLMs as router-decoupled mixture of experts with system co-design. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 116126–116148, 2024. arXiv:2410.19123.
- [8] Pei Cao, Edward W. Felten, Anna R. Karlin, and Kai Li. A study of integrated prefetching and caching strategies. In *Proceedings of ACM SIGMETRICS*, 1995.
- [9] Shiyi Cao, Shu Liu, et al. MoE-Lightning: High-throughput MoE inference on memory-constrained GPUs. *arXiv preprint arXiv:2411.11217*, 2024.
- [10] Martin C. Carlisle and Errol L. Lloyd. On the k -coloring of intervals. *Discrete Applied Mathematics*, 59, 1995.
- [11] Hongtao Chen, Weiyu Xie, Boxin Zhang, Jingqi Tang, Jiahao Wang, Jianwei Dong, Shaoyuan Chen, Ziwei Yuan, Chen Lin, Chengyu Qiu, Yuening Zhu, Qingliang Ou, Jiaqi Liao, Xianglin Chen, Zhiyuan Ai, Yongwei Wu, and Mingxing Zhang. KTransformers: Unleashing the full potential of CPU/GPU hybrid inference for MoE models. In *Proceedings of the 31st ACM Symposium on Operating Systems Principles (SOSP)*, 2025.

- [12] Liangkun Chen, Zijian Wen, Tian Wu, Xiaoxi Zhang, and Chuan Wu. SP-MoE: Speculative decoding and prefetching for accelerating MoE-based model inference. *arXiv preprint arXiv:2510.10302*, 2025.
- [13] csantiago78. llama.cpp pull request #27861: GPU-resident LRU cache for host-offloaded MoE expert weights. <https://github.com/ggml-org/llama.cpp/pull/27861>, 2026. Draft, accessed September 2026.
- [14] Artyom Eliseev and Denis Mazur. Fast inference of mixture-of-experts language models with offloading. *arXiv preprint arXiv:2312.17238*, 2023.
- [15] Zhiyuan Fang, Zicong Hong, Yuegui Huang, Yufeng Lyu, Wuhui Chen, Yue Yu, Fan Yu, and Zibin Zheng. Fate: Fast edge inference of mixture-of-experts models via cross-layer gate. *arXiv preprint arXiv:2502.12224*, 2025.
- [16] Chris Fawcett and Holger H. Hoos. Analysing differences between algorithm configurations through ablation. *Journal of Heuristics*, 22:431–458, 2016.
- [17] Georgi Gerganov and contributors. llama.cpp: LLM inference in C/C++. <https://github.com/ggml-org/llama.cpp>, 2023–2026.
- [18] Aman Gupta. llama : add a GPU cache for MoE experts kept in host memory. llama.cpp pull request 29887, <https://github.com/ggml-org/llama.cpp/pull/29887>, 2026. Opened 3 October 2026.
- [19] Duc Hoang, Ajay Jaiswal, Mohammad Samragh, and Minsik Cho. SpecMD: A comprehensive study on speculative expert prefetching. In *International Conference on Machine Learning (ICML)*, 2026. arXiv:2602.03921.
- [20] Torsten Hoefer, Dan Alistarh, Tal Ben-Nun, Nikoli Dryden, and Alexandra Peste. Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks. *Journal of Machine Learning Research*, 22(241), 2021.
- [21] Torsten Hoefer and Roberto Belli. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2015.
- [22] En-Ming Huang, Li-Shang Lin, et al. Efficient CPU-GPU collaborative inference for MoE-based LLMs on memory-limited systems. *arXiv preprint arXiv:2512.16473*, 2025.
- [23] Ranggi Hwang, Jianyu Wei, et al. Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024. arXiv:2308.12066.
- [24] Andrei Ivanov, Nikoli Dryden, Tal Ben-Nun, Shigang Li, and Torsten Hoefer. Data movement is all you need: A case study on optimizing transformers. In *Proceedings of Machine Learning and Systems (MLSys)*, 2021.
- [25] Akanksha Jain and Calvin Lin. Back to the future: Leveraging Belady’s algorithm for improved cache replacement. In *Proceedings of the 43rd International Symposium on Computer Architecture (ISCA)*, 2016.
- [26] Akanksha Jain and Calvin Lin. Rethinking Belady’s algorithm to accommodate prefetching. In *International Symposium on Computer Architecture (ISCA)*, 2018.
- [27] Raj Jain. *The Art of Computer Systems Performance Analysis*. Wiley, 1991.
- [28] Yinsicheng Jiang, Yao Fu, Yeqi Huang, et al. MoE-CAP: Cost-accuracy-performance benchmarking for mixture-of-experts systems. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2412.07067.
- [29] Tomas Kalibera and Richard Jones. Rigorous benchmarking in reasonable time. In *International Symposium on Memory Management (ISMM)*, 2013.
- [30] Keisuke Kamahori, Tian Tang, et al. Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models. *arXiv preprint arXiv:2402.07033*, 2024.
- [31] Byeongju Kim, Jungwan Lee, Donghyeon Han, Hoi-Jun Yoo, and Sangyeob Kim. FlashMoE: Reducing SSD I/O bottlenecks via ML-based cache replacement for mixture-of-experts inference on edge devices. *arXiv preprint arXiv:2601.17063*, 2026.
- [32] leloch. MoE expert cache for llama.cpp: VRAM caching of hot CPU-resident experts with hybrid hit/miss execution. <https://github.com/leloch/llama.cpp/tree/moe-cache-v2-pr>, 2026. Branch moe-cache-v2-pr, commit e3096b0 (6 August 2026). Proposed upstream as llama.cpp pull request #24524 and RFC discussion #24528; not merged. Accessed September 2026.
- [33] leloch. RFC: MoE expert cache, VRAM caching of hot CPU-resident experts with hybrid hit/miss execution. llama.cpp discussion #24528, <https://github.com/ggml-org/llama.cpp/discussions/24528>, 2026. Posted 12 June 2026; accessed September 2026.
- [34] Shuhuai Li, Jianghao Lin, Dongdong Ge, and Yinyu Ye. MoE-SpAc: Efficient MoE inference based on speculative activation utility in heterogeneous edge scenarios. *arXiv preprint arXiv:2603.09983*, 2026.
- [35] Ye Li, Jian Tan, Bin Wu, Xiao He, and Feifei Li. ShapleyIQ: Influence quantification by Shapley values for performance debugging of microservices. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Volume 4, pages 287–323, 2024.
- [36] Fangzhou Liang, Yibin Shen, Jianmin Hu, Jiayang Xu, Hanchi Gao, Minxian Xu, and Zili Meng. SSD-LLaMA: SSD-native inference for trillion-parameter MoE at 1+ token/s on a consumer PC. *arXiv preprint arXiv:2609.18110*, 2026.
- [37] Jingcong Liang et al. Not all models suit expert offloading: On local routing consistency of mixture-of-expert models. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2505.16056.
- [38] Yu Lin, Yiming Wang, Runyuan Cai, Hanze Liu, and Xiaodong Zeng. The other half of the memory wall: Serving 35B MoEs from SSD with trained routing prediction. *arXiv preprint arXiv:2609.18063*, 2026.
- [39] milto22. llama.cpp pull request #26563: Expert caching that greatly increases performance (self-contained, off by default). <https://github.com/ggml-org/llama.cpp/pull/26563>, 2026. Accessed September 2026.
- [40] OpenAI. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025.

- [41] Anish Saxena, Po-An Tsai, Hritvik Taneja, Aamer Jaleel, and Moinuddin Qureshi. Cascade: Utility-driven speculative decoding for mixture-of-experts. In *Proceedings of Machine Learning and Systems (MLSys)*, 2026. arXiv:2506.20675 (2025).
- [42] Musa Shams. Paging the experts: A reproducible characterization of flash-backed MoE inference on iPhone. *arXiv preprint arXiv:2609.29032*, 2026.
- [43] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. FlexGen: High-throughput generative inference of large language models with a single GPU. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*, pages 31094–31116, 2023. arXiv:2303.06865.
- [44] Xiaoni Song, Zihang Zhong, et al. ProMoE: Fast MoE-based LLM serving using proactive caching. *arXiv preprint arXiv:2410.22134*, 2024.
- [45] Zhenyu Song, Daniel S. Berger, Kai Li, and Wyatt Lloyd. Learning relaxed Belady for content distribution network caching. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2020.
- [46] Peng Tang, Jiacheng Liu, et al. HOBbit: A mixed precision expert offloading system for fast MoE inference. *arXiv preprint arXiv:2411.01433*, 2024.
- [47] Aditya Ukarande, Deep Shekhar, Marc Blackstein, and Ram Rangan. Efficient, VRAM-constrained xLM inference on clients. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 8, 2026. Industry track. arXiv:2604.26334. Artifact: <https://github.com/deepsnvn/pipeshard-mlsys26-ae>, tag v2.0.3-mlsys26.
- [48] Wenfeng Wang, Jiacheng Liu, Xiaofeng Hou, Xinfeng Xia, Peng Tang, Mingxuan Zhang, Chao Li, and Minyi Guo. MoE-SpeQ: Speculative quantized decoding with proactive expert prefetching and offloading for mixture-of-experts. *arXiv preprint arXiv:2511.14102*, 2025.
- [49] Wenxin Wang, Yule Hou, et al. Achieving cloud-grade SLOs for local mixture-of-experts inference through CPU-GPU hybrid design. *arXiv preprint arXiv:2606.10493*, 2026.
- [50] Zihan Wang, Yuqi Wang, et al. SeqMoE: Toward full-load performance via predictive and graph-compatible MoE offloading. *arXiv preprint arXiv:2609.12978*, 2026.
- [51] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- [52] Leyang Xue, Yao Fu, et al. MoE-Infinity: Efficient MoE inference on personal machines with sparsity-aware expert cache. *arXiv preprint arXiv:2401.14361*, 2024.
- [53] Sanjali Yadav and Bahar Asgari. Mira: Memory-efficient MoE inference using adaptive caching and predictive expert staging. *arXiv preprint arXiv:2609.38090*, 2026.
- [54] An Yang et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [55] Shuo Yang, Xiaoze Fan, Melissa Pan, Haocheng Xi, Zhe Wang, Shanlin Sun, Kurt Keutzer, Song Han, Matei Zaharia, Chenfeng Xu, and Ion Stoica. FreeToken: Efficient edge-native MoE serving with bandwidth-adaptive execution. *arXiv preprint arXiv:2608.16157*, 2026.
- [56] Yichao Yuan, Lin Ma, et al. MoE-Lens: Towards the hardware limit of high-throughput MoE LLM serving under resource constraints. *arXiv preprint arXiv:2504.09345*, 2025.
- [57] Zhang, Gao, and Mitra. SAEM: Stage-aware expert management for memory-efficient MoE inference in chain-of-thought reasoning, 2026. arXiv:2608.21614.
- [58] Hanhaodi Zhang. Budgeting bytes: A windowed storage roofline and dual-budget architecture ablations for storage-bound LLM decoding. *arXiv preprint arXiv:2609.04238*, 2026.
- [59] Jiamu Zhang, Liang Wu, Mayank Darbari, and Liangjie Hong. WiSP: A working-set view of mixture-of-experts serving on extremely low-resource hardware. *arXiv preprint arXiv:2606.21868*, 2026.
- [60] Lei Zhang, Reza Karimi, Irfan Ahmad, and Ymir Vigfusson. CHOPT: Optimal data placement in a hybrid memory system. *Proc. ACM Meas. Anal. Comput. Syst. (SIGMETRICS)*, 2020.
- [61] Lei Zhang, Reza Karimi, Irfan Ahmad, and Ymir Vigfusson. Optimal data placement for heterogeneous cache, memory, and storage systems. *Proceedings of the ACM on Measurement and Analysis of Computing Systems (PO-MACS)*, 4(1):6:1–6:27, 2020.
- [62] Yu Zhang. Reproducible evaluation of MoE expert caching: Replay semantics, workload contamination, and operating regimes. *arXiv preprint arXiv:2608.07911*, 2026.
- [63] Yujie Zhang, Shivam Aggarwal, and Tulika Mitra. DAOP: Data-aware offloading and predictive pre-calculation for efficient MoE inference. In *Design, Automation & Test in Europe Conference (DATE)*, pages 1–7, 2025. arXiv:2501.10375.
- [64] Shuzhang Zhong et al. HybriMoE: Hybrid CPU-GPU scheduling and cache management for efficient MoE inference. In *Design Automation Conference (DAC)*, 2025. arXiv:2504.05897.
- [65] Shien Zhu, Samuel Bohl, et al. Pre-attention expert prediction and prefetching for mixture-of-experts large language models. *arXiv preprint arXiv:2511.10676*, 2025.
- [66] Zeyu Zhu, Gang Li, Peisong Wang, Zitao Mo, Minnan Pei, Zhuoran Song, Xiaoyao Liang, and Jian Cheng. DALI: A workload-aware offloading framework for efficient MoE inference on local PCs. *arXiv preprint arXiv:2602.03495*, 2026.

A Scientific benchmarking checklist

How this paper follows the twelve rules of Hoefer and Belli [21].

Rule	Where and how
1. Report the base case of a speed-up and its absolute performance.	Every ratio in Table 2 is next to both absolute speeds; llama.cpp is configured with per-layer expert offload at equal expert memory, the setting its users run.
2. Explain subsets of benchmarks or resources.	Two models chosen for size and precision (MXFP4, BF16), a third that is FreeToken’s own headline model; budgets span what fits a 32 GB card; the 43.75% budget is omitted where it does not fit. All 30 AIME-25 problems are used.
3. Arithmetic mean for costs, harmonic mean for rates.	The accounting, the panel and the value of foresight average time per token over problems (a cost) and quote speeds as the harmonic mean this implies; in Table 2 every problem decodes the same number of tokens, and ratios of total time agree with the reported ratios of mean rates within 0.003.
4. Avoid summarising ratios.	Ratios of means are never averaged across budgets or models; ranges list the per-budget values, and the medians reported across models (Table 13) or across published rows (Appendix J) are medians of per-cell quantities, labelled as such.
5. Report whether measurements are deterministic; give confidence intervals.	Decode is greedy and deterministic per engine; timing is not. Every comparison has a 95% interval over problems; the panel (job 099) adds intervals over hosts, the unit that varies most.
6. Do not assume normality.	Intervals are bootstrap percentile intervals (10,000 resamples, paired by problem); no distributional assumption.
7. Compare nondeterministic data soundly.	Paired bootstrap of the ratio of means within a host; across the panel’s hosts, a two-stage bootstrap (hosts, then problems) [29]; the variant a system is scored with is picked on a separate launch (except the Qwen3 25% row of Table 2).
8. Check that central tendency is the right summary.	Mean speed over 256 tokens is FreeToken’s own metric; per-token p50 and p99 latencies are recorded for every problem in the released rows.
9. Document all factors and the setup.	Machines, probes, power limits, memory clocks (Section 2), software commits (llama.cpp 4da6337, FreeToken 0d652e7), flags and job scripts are released; Appendix K.
10. Report measurement, synchronisation and summarisation.	One client times every system of Table 2 from streamed token arrivals; the oracle comparisons of Section 4 are timed in process by <code>llama-ec-bench</code> over teacher-forced text, every policy by the same timer (Section 2).
11. Show upper performance bounds.	The bound (Section 3) is a column of Table 2; Table 16 tightens it four ways; Section 3 applies it to the published literature and to our cache with the same procedure.
12. Plot enough to interpret; connect points only for trends.	Figure 1 plots every host’s ratio with its interval; Fig. 3 connects measured windows for the trend only and marks engine measurements separately; Figs. 6 and 8 are bar charts and Figs. 4, 5 and 9 scatter plots.

B Prediction log

Each job’s predictions are in the header of its script on the public `gpu` branch, committed and pushed before the machine started; git history orders every prediction before its measurement. Outcome notes with the statistics are in `prereg/`. The scorecard, generated from `prereg/scorecard_clauses.json`, splits every prediction of jobs 073–098 into its separable clauses and scores each under one rule: *held* only when the point estimate is on the predicted side and the 95% paired-bootstrap interval excludes the threshold (for a band, lies inside it); *held (point)* when only the point estimate is, or no interval exists (except in the hand scoring of jobs 073–098, which holds deterministic counts such as reads outright, since they have no sampling error); *failed* when the point estimate is on the wrong side;

untested when the clause could not be evaluated; *void* when the job was voided. Of 565 clauses, 218 held, 193 held on the point estimate, 142 failed, 8 were untested and 4 void: 7 of 19 scored clauses held in jobs 073–075, when the system was being built, 19 of 23 in jobs 076–081, 11 of them sign-only predictions made after a same-host pilot, 24 of 38 in jobs 082–087, and 168 of 473 in jobs 088 onward, where 42 of 82 sign clauses held (75 on the point estimate) and 90 of 301 bands failed. Most bands that failed are the sizes of effects whose direction held: our slow-link lead landed at $1.51\text{--}1.58\times$ against a predicted $1.10\text{--}1.50$; the rerun of Table 2 on a second host (job 089) gave ratios $0.06\text{--}0.11$ below host B’s at five of six cells against a predicted ± 0.06 , host-bound speeds $14\text{--}17\%$ lower against a predicted $\pm 12\%$, and LRU $6\text{--}8\%$ slower than decayed frequency at the two GPU-heavy cells against a predicted $10\text{--}30\%$; on the 4090 and 3090 hosts (jobs 091, 092; Appendix D) our lead over FreeToken and its fraction of the limit overshot their bands, FreeToken’s carried-over backend fell far below its band, and two launch-order checks exceeded 3% (7% and, at gpt-oss 11% on the 3090 host, 11%); in the first oracle job (093; Section 4) the oracle’s gain fell outside $20\text{--}80\%$ at the two lowest budgets (it lost), its speed fell just below the predicted $45\text{--}65\%$ of the limit at all four host-bound cells ($43.7\text{--}44.8\%$), its hit rate and admissions missed the optimum’s by more than the predicted 3 points and 30% at three and two of the four, no-overlap cost 0% against a predicted $5\text{--}25\%$, and all-CPU cost 15% on Qwen3 against a predicted $0\text{--}10\%$; in the bytes-oracle job (094) the bands missed as Appendix F describes; in the single-read job (095) 44 of 56 clauses held, and the 12 that failed are of size: the model on the fetch oracle’s counters was predicted within 10% and missed by $13\text{--}17\%$ at four cells, the fetch oracle’s reads were predicted within 10% of the simulation’s and came 17% below at the two GPU-bound cells (the registered simulator left a resident carried over from the previous sequence with no next use, and so evicted it first; corrected, the simulation is within 4% of the engine’s reads at every cell, Appendix F), the prefetching oracle was predicted at $55\text{--}85\%$ of the limit at the host-bound cells and reached 44 and 51% at the two 25% cells, the model’s over-prediction of it was predicted at $15\text{--}50\%$ and came to 7 and 13% at the two lowest budgets, and the paced variant, predicted within 10% of the unpaced one, was $12\text{--}13\%$ faster at the two 25% cells; in the confirmatory factorial (096) the clauses held at most cells of both hosts and the failures are listed in Appendix F, the largest being the difference between the hosts; and in the FreeToken tuning job (098) its best setting stayed within the predicted 10% of its default and our ratios within the predicted bands, while the fixed fetch caps, the thread count and the CPU-only comparison missed as `prereg/freetoken.outcome_098.md` describes. Among the 23 threshold failures are ours at $1.56\text{--}1.78\times$ llama.cpp on the slow hosts against a predicted 1.8 , the Mixtral tie against a predicted $1.3\times$, the hit-optimal oracle’s gain at the two GPU-bound cells (48 and 56%) against a predicted ceiling of 15% , the bytes-optimal oracle short of beating the hit-optimal one by 15% at the two lowest budgets, and its 16-token window short of keeping 70% of its gain at three cells, fetch’s gain differing between hosts O4 and O5 by more than 0.15 at five cells, Belady’s single-read prefetch above $1.05\times$ the online policy at Qwen3 12.5% on O4, and FreeToken’s CPU backend running close to stock llama.cpp’s where we predicted at least $2\times$ and $1.2\times$; the 3 equality failures are two of the 4090 host’s FETCH tables, which copy one more expert than predicted, and FreeToken’s best backend at gpt-oss 25% in job 098 (offload, not hybrid); the 7 sign failures are LRU behind FreeToken at gpt-oss 11% , where we predicted it would still lead, and the hit-optimal oracle slower than the online policy at gpt-oss 11% and Qwen3 12.5% . Against the running log kept during the study, two clauses are downgraded (a llama.cpp clause of job 074b that the log counted as held was never run; job 085b’s own restatement of its fourth prediction, “at least 2 of 3”, failed at 1 of 3, though the pooled 4 of 6 held), job 073’s “identical outputs” is scored on fully identical outputs ($0\text{--}3\%$) rather than on agreement of the first 160 characters ($27\text{--}47\%$), and ten clauses the log counted as held, among them job 087’s first, hold on the point estimate only under the interval rule; `prereg/scorecard.outcome.md` lists every change. Job 085’s Qwen3 half ran as 085b on a second machine with a slow link after a hung conversion. Job 084 ran on a throttled card and was rerun, with its predictions unchanged, as 084b/084c. The batched-verification variants of Section 5 (rejected drafts free, routed like the true token, or routed at random) are in `prereg/batchk.outcome.md`.

The clauses were scored by hand from the outcome notes. A script (`scripts/scorecard.auto.py`) re-scores every clause whose measured value is a number and whose threshold parses, 469 of 565: it agrees with the hand score on 432; of the 37 differences, 36 are the deterministic counts (reads, divergences, table entries) that the hand scoring holds outright and the script scores held (point), and 1 is a boundary value the hand scoring read unrounded (1.249 against a band starting at 1.25). Of the clauses the script scores held on the point estimate, 213 have no interval and 7 have one that crosses the threshold. Job 099’s clauses are scored by machine from the start (`scripts/panel_099.py`): of 273, 190 held, 42 held on the point estimate, 39 failed and 2 were untested (`prereg/scorecard_099.json`); the two untested clauses are one host’s replay check, whose step failed when a Python package did not install. The failures concentrate on the two hosts whose link reads at less than half their CPU’s rate, where the bands, set from four earlier hosts none of which was link-starved, did not encode host dependence; and on the planning time of the noisy whole-future window (`prereg/panel.outcome_099.md`). Job 100 (`scripts/job100.py`): of 216, 41 held, 122 held on the point estimate, 27 failed and 26 were untested. Most failures are one prediction, that the deployed-path windows miss within 10% as often as a replay that admits at once (their copies land late); the rest are the probe-only model on the half-width-link host and four band or ordering edges. The untested clauses are the planning time of the

deployed-path windows, which the engine does not time, and the paired prediction for a host that never reached a timed step (`prereg/window_outcome_100.md`). Job 101 (`scripts/job101.py`; the lowest-ratio panel machine relaunched, and a second Ryzen 7 9800X3D behind a slower link): of 25, 6 held, 19 held on the point estimate, 0 failed and 0 were untested (`prereg/crossover_outcome_101.md`). Job 102 (`scripts/job102.py`; the read-schedule microbenchmark on three hosts): of 33, 0 held, 28 held on the point estimate (none has an interval), 5 failed and 0 were untested. Four failures are one prediction, that waiting per layer would cost at least 0.03 of the host term against the analytic schedule; on two of the three hosts it costs nothing measurable. The fifth is the ordering of the two budgets on the Intel host (`prereg/readsched_outcome_102.md`).

The clause-by-clause tables (565 rows for jobs 073–098, 273 for job 099, 216 for job 100, 25 for job 101, 33 for job 102) are in the supplement (`paper/supplement.pdf`), generated by the same scripts.

C The calibrated time model

Our cache’s one per-machine decision, how many of a layer’s misses to fetch rather than run on the CPU, comes from a calibrated model of time per token. With X_c the bytes the CPU reads per token and X_p the bytes the link reads,

$$T = G + \max\left(\frac{X_c}{B_c}, \frac{X_p}{B_p}, \frac{X_c + X_p}{B_{cp}}\right), \quad (2)$$

with one constant G per engine and model, fitted once on one host and frozen. It is a calibrated model, not a law: G is fitted, and it exceeds the whole all-in-VRAM token (5.17 against 3.92 ms) because it absorbs per-token host work and the CPU helpers’ shortfall against the probe (below). As a predictor of time it ties a naive one-path model with the same G (time = G + all host bytes over the best probed rate):

Held-out set	Eq. (2)		one path	
	median	90th pct.	median	90th pct.
blind test, 33 rows on 7 hosts	3.4%	12%	3.6%	31%
frozen, 30 rows on 4 later hosts	5.6%	16%	6.1%	11%

It over-predicts Qwen3’s speed on the later hosts by +5 to +21%, a budget-dependent bias whose G was borrowed from llama.cpp. Its value is the per-machine fetch table it picks (Eq. (3) below), which gains 3.3–8.0% over a fixed table on host B and 9–34% on hosts with half the link (Appendix I).

The blind test in full. On every machine of the blind test the job wrote the model’s prediction from its own probe before any model ran (Table 7; all of its cache rows are gpt-oss-120b on RTX 5090 hosts). Over 29 configurations on 7 machines (33 measurements; one machine ran twice) the median error is 3.7% per configuration; without the 12-channel EPYC 9655 it is 3.3% over 24 configurations (90th percentile 7.5%). llama.cpp `--n-cpu-moe` is predicted within 4% on the Ryzen hosts, including two Qwen3 configurations on host B (+3.6%, +4.0%; for Qwen3, $G=4.3$ ms, the mean residual of three llama.cpp runs on another host), and misses by +10–+119% on the hybrid-core and server hosts. With its constants frozen, the model scores the cache on the 4 RTX 5090 hosts rented later (30 measurements at the six cells of Table 2, not written before the runs) at a median 5.6%, 90th percentile 16%, worst 21% (Qwen3-30B-A3B 43.75% on oracle host O1). Refit under leave-one-host-out validation, the token-level max has a median error of 3.1% (90th percentile 9.4%), the additive form $G + X_c/B_c + X_p/B_p$ 4.5% (11.3%) and a per-layer max summed over layers 3.2% (10.2%); the max beats the sum on the three rows that copy misses on demand behind a 53 GB/s link, where both paths carry most of the bytes, and on the EPYC 9655 rows, where both fail, the sum is closer on the two rows that copy behind a slow link, and elsewhere the two are within a few points (Appendix C). Each failure has a measured cause (Appendix C): on-demand copies behind a slow link share DRAM less evenly than the probe’s combined rate says; llama.cpp splits every matrix evenly across threads and runs at 46% of the model on a hybrid-core host; and on a 12-channel EPYC host memory stops being the limit and a per-layer latency floor takes over. G (5.17 ms refit) exceeds the whole all-in-VRAM token (3.92 ms) because it absorbs a shortfall the probe does not see: the helpers read at $1.09\text{--}1.31\times$ the probe’s time per expert, and a fitted rate factor is collinear with G at this sample size.

What the model says, and the split it picks. Equation (2) is a calibrated model of execution that reads host memory in sequence with its GPU work, as our cache and llama.cpp do. For such an engine every mechanism acts only through the bytes read from host memory per token and the number of read paths in use: a second path adds bandwidth on every host of the blind test but the 12-channel EPYC (B_{cp} against $\max(B_c, B_p)$): 8–17% on its Ryzen

Table 7: The calibrated model predicted blind: each machine wrote its prediction from its own bandwidth probe, with constants frozen in the public repository, before any model run. Error of predicted over measured speed, gpt-oss-120b (cache rows) and llama.cpp --n-cpu-moe (last column, %).

Host CPU	Job	n	Median err %	Max err %	llama.cpp err %
Core Ultra 7 270K (8P+16E)	069c	5	1.5	11.9	+119
EPYC 7352 (Zen 2)	069c	5	3.2	11.7	+10
EPYC 9655 (12-ch. DDR5)	069c	5	18.7	31.9	+16
Ryzen 9 7900	072	5	4.9	8.2	+4
Ryzen 9 9950X3D (A)	073	8	1.6	4.5	—
Ryzen 9 7950X	074	1	4.0	4.0	—
Ryzen 9 9950X	076	4	2.5	5.0	—
Ryzen 9 9950X3D (B)	082	0	—	—	+4, +4

hosts, 1% on the Core Ultra, 33% on the Zen 2 EPYC; the RTX 3090 host’s Core i9-11900KF, where the two paths together read less than the CPU alone, is a second exception), and that is the whole gain available to fetching or splitting misses between the CPU and PCIe. Prefetching never reduces the bytes; what it can buy is overlap of the reads with the step, which the model does not price: charging every read in sequence, it over-predicts the time per token of MIN prefetched by +1 to +38% (Section 4) and of a Belady prefetch by up to 53% (Appendix F). The model’s job in the system is the split. Its per-layer form,

$$T(n, f) = \max\left(\frac{fS}{B_p} + g, [n > f](L_c + \frac{(n-f)S}{B_c}), [0 < f < n] \frac{nS}{B_{cp}}\right), \quad (3)$$

with g the GPU’s expert work per layer ($48 \mu s$ for Qwen3, $37 \mu s$ for gpt-oss, from profiles) and $L_c = 20 \mu s$ the hand-off, picks for each count n of missed experts the number f to copy with the smallest T . On host B the model’s table beats the fixed table we used before at every budget of both models, by 3.3–8.0%; on hosts whose link we measured at 27–30 GB/s instead of 53 it fetches less and gains 21–34% on gpt-oss and 9–18% on Qwen3, the per-machine split mattering most where the link is slow (Appendix I). In the running example on host B the table is 0,0,1,1,2 ($f(0..4)$): one missed expert runs on the CPU, hidden behind the GPU’s other experts; from two misses the model copies one. The lead over FreeToken at that cell, $1.27\times$, is mostly the admission policy (77% of it by the LRU ablation of Appendix I), the rest the CPU path that FreeToken’s bandwidth model declines to use there.

Is the max the right form? In 19 of the 29 configurations nothing is copied, so they test one roof. We therefore refit the model and its alternatives on the same 33 measurements under leave-one-host-out validation (each of the 7 hosts predicted with constants fitted on the other six): the token-level max of Eq. (2) with G refit has a median error of 3.1% (90th percentile 9.4%); the additive form $G + X_c/B_c + X_p/B_p$, 4.5% (11.3%); a per-layer max summed over layers from the engine’s per-layer miss histograms, 3.2% (10.2%). The additive form ties in sample but misses the three rows that copy misses on demand behind a 53 GB/s link, where both paths carry most of the bytes, by -11%, while the max is within $\pm 0.2\%$ there; on the slow-link copy rows the additive form is the closer of the two. The per-layer form neither beats the max nor explains its failures: the Jensen gap between a sum of per-layer maxima and the maximum of sums is a fifth of the Core Ultra’s residual, none of the EPYC 7352’s, and has the wrong sign on the EPYC 9655 and on the prefetch rows. We keep Eq. (2), with its evidence stated: the max beats the sum on the three fast-link copy rows and on the EPYC 9655 rows, where both fail, the sum is closer on the two slow-link copy rows, and elsewhere the two are within a few points, so the form is settled only where both paths carry most of the bytes. FreeToken’s own runs are not predicted, because its counters are not exposed; the model covers our engine and llama.cpp.

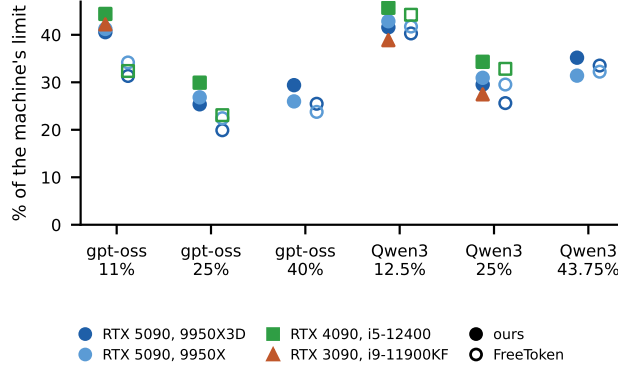


Figure 4: Ours (filled) and FreeToken (hollow) as a percentage of each machine’s own bound (exact optimum, the machine’s highest probed host rate, the card’s datasheet rate) at every cell of Table 8: two RTX 5090 hosts, an RTX 4090 and an RTX 3090 (FreeToken’s 3090 cells, its carried-over backend running below llama.cpp, are omitted). The limit ranges from 41 to 519 tok/s across these cells; ours stays at 25–46% of it.

Where it fails, and why. Each failure now has a measured cause. (i) Copying misses on demand (FETCH) at 25% is over-predicted by +11.7% on the Core Ultra 7 and +11.4% on the EPYC 7352 (links of 36 and 26 GB/s); +8.8% and +8.5% with G refit. The engine’s per-layer timers show why: a CPU miss served beside a copy takes 20–62% longer than one served alone, so the two paths share DRAM less evenly than the probe’s combined rate says; a variant of the model that uses the probe’s per-path rates under contention and a per-layer hand-off constant predicts both rows within +3.1% and +4.1%, at a worse median (4.2%). (ii) llama.cpp on a Core Ultra 7 with 8 performance and 16 efficiency cores runs at 46% of the model’s prediction at its default thread count, because it splits every matrix evenly across its threads; our helpers claim work dynamically and stay on the model on the same machine. We did not test llama.cpp restricted to the performance cores. (iii) On a 12-channel EPYC 9655 (559 GB/s) host memory stops being the limit (+16 to +29% with G refit under leave-one-host-out validation; Table 7 gives the blind errors): the per-layer timers show a floor of 55–70 μ s per layer with CPU misses that no desktop host has (their intercepts are at or below zero), the latency of the hand-off at a bandwidth where a 13 MB expert takes 24 μ s. (iv) Prefetch configurations are under-predicted by -8.7 to -3.4%: their copies overlap attention, which the model charges in full.

D The grid: other cards, and the scope of the gain

Table 8 lists every cell of Table 2’s protocol on the four machines we measured, each against its own limit, and Fig. 4 plots the fractions. The protocol was repeated on an RTX 4090 next to a Core i5-12400 (the host reads 37 GB/s, the link 25) and an RTX 3090 next to a Core i9-11900KF (45 GB/s; CPU and link together read *less* than the CPU alone, so the model’s table copies nothing), at the cells that fit 24 GB. On the 4090 ours runs 1.03–1.37 $\times$ FreeToken and 1.8–2.6 $\times$ llama.cpp at half host B’s absolute speeds, the host being the reason; on the 3090 it runs 1.6–2.2 $\times$ llama.cpp (FreeToken’s carried-over backend ran below llama.cpp there and is not counted). The cards’ own bandwidth never enters: every one of the seven cells on the 24 GB cards is host-bound at the limit, and ours stands at 30–46% and 27–42% of it. Mixtral-8x7B (8 experts per layer, top-2; 26 GB of experts on the 3090) marks the scope: with 2 of 8 experts resident the cache hits 31% of reads where whole-layer pinning hits 25%, so there is little locality to earn, and ours ties llama.cpp (1.015 [1.008, 1.024]); on the 128-expert models the same cache hits 53–79% at 11–25% resident. Across eight hosts the gpt-oss lead over FreeToken grows with the host’s CPU-to-link ratio, from 1.09–1.22 $\times$ where the CPU reads 1.05–1.14 $\times$ its link’s rate to 1.51–1.58 $\times$ behind a link of half the bandwidth, in the direction the model’s split implies (the model does not predict FreeToken’s runs), and the Qwen3 lead stays at 0.95–1.15 $\times$ (Fig. 5, Appendix I).

The 24 GB cards run the cells that fit (the VRAM arithmetic, written on the machine from the GGUF header, skipped Qwen3 at 43.75%: 28.0 GB of slots, dense weights and KV against a 23 GB limit). The predictions for jobs 091 and 092 and their outcomes are in `prereg/grid_outcome_091.md` and `092.md`: on the 4090 our lead over FreeToken at gpt-oss 11% (37%) exceeded the predicted 10–35% band, ours ran 1.78 $\times$ llama.cpp at Qwen3 12.5% against a predicted 1.8, the model’s tables copied one more expert than host B’s at one entry each (the prediction reasoned from the link alone; the CPU is slower in the same proportion), and ours’ fraction of the limit at Qwen3 12.5% (45.7%) exceeded the predicted 25–45%; on the 3090 FreeToken’s carried-over backend ran below llama.cpp,

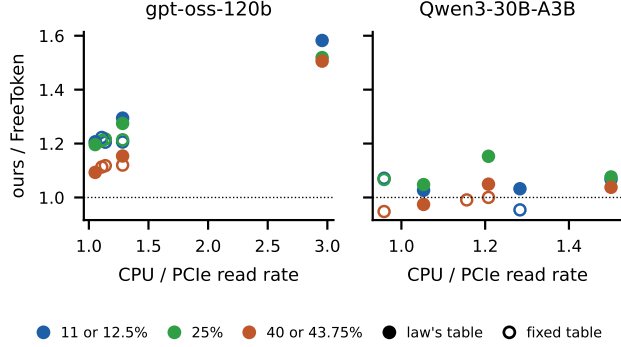


Figure 5: Ours over FreeToken’s better backend (carried over from host B on host S) in every same-machine comparison of the current engine on the two models of Table 2 (eight hosts), against the machine’s CPU-to-PCIe read ratio (FreeToken’s own probe). Filled: FETCH table from the model on that machine; hollow: the fixed table. The rightmost gpt-oss points are a Core i9-14900K with FreeToken’s executor on its performance cores; the rightmost Qwen3 points are a Ryzen 9 7900.

ours ran $1.56\text{--}1.64\times$ llama.cpp at two cells against a predicted 1.8, Mixtral tied at $C=2$ against a predicted $1.3\times$ and its $C=4$ cell was lost to the job’s deadline, and one launch-order check exceeded 3% (7%).

E The accounting as a model

Between the measured time of a cell and its limit lie five modelled fixes: overlap of host reads with GPU work (in Eq. (1)’s form, which also rebalances experts onto the CPU), the optimum’s reads instead of the online policy’s (*foresight*), the best online policy’s reads at the probe’s highest rate instead of the engine’s through Eq. (2) (policy and read paths), the engine’s per-token host work, and GPU kernels at the datasheet rate. Adding them in a fixed order makes the first one added look largest, so we report each fix’s Shapley value, its average marginal cost over every order (Fig. 6), a method with a precedent in performance debugging of microservices [35]. Two of the five terms are definitions rather than measurements: the GPU term is the residual, its factor calibrated per cell so that the model with no fix applied reproduces the measured time, and “overlap” is Eq. (1)’s form. On host B the model has two regimes. In the four host-bound cells missing foresight is the largest cost, 37–53% of the gap (26–31% of the token; in the running example 37%, where it edges overlap by 0.14 ms and is the largest marginal cost in 70 of the 120 orders); in the two GPU-bound cells host reads serialised with GPU work is largest, 39–46% of the gap, and foresight second at 23%. Policy and read paths cost 12–16%, per-token host work 3–10%, and GPU kernels below datasheet, net of the overlap the engine achieves, 4–16%. The regimes are a property of the host: recomputed for the three oracle hosts, whose host memory is weaker, foresight edges overlap at three of their six GPU-bound cells. The model prices foresight and overlap as nearly independent players, and the interior states it values are not realised by any online policy. The rest of this section measures them.

F Foresight in the engine

Four jobs ran oracle policies in the engine, each on a rented RTX 5090 host with weaker host memory than host B’s. The fourth (096) is the confirmatory factorial of Fig. 1: ten configurations per cell, each in a fresh context, in an order shuffled per cell with a recorded seed, on hosts O4 (Ryzen 9 9950X: 43, 45 and 51 GB/s by the CPU, over the link and at best) and O5 (Ryzen 9 9950X3D: 47, 27 and 51 GB/s), with ten predictions per host committed before either started (`prereg/foresight_outcome.096.md`). They held at most cells. The failures: fetch’s gain above the predicted band at Qwen3 25% on O4 and different between the hosts by up to 0.26 against the 0.15 predicted; Belady’s single-read prefetch faster than predicted at Qwen3 12.5% on O4, with reads below the predicted $1.25\text{--}1.9\times$ the MIN prefetch’s at Qwen3 12.5% on both hosts and at gpt-oss 11% on O4 (down to $1.09\times$); the bytes share at Qwen3 12.5% on O4 (60% of the gap against a predicted 25–55%); and the reads fetch-on-admit saves at gpt-oss 40% on O5, just above the predicted 22%. Reading the online policy’s admissions once came to $0.97\text{--}1.03\times$ at the host-bound cells, at the bottom of the predicted $0.97\text{--}1.12\times$ although its reads fell as predicted, and to $0.95\text{--}1.02\times$ at the GPU-bound cells. Table 9 lists every cell of the single-read factorial of Section 4 on host O3 (a Ryzen 9 9950X whose probe reads

Table 8: The grid: Table 2’s protocol on every card and host we measured, each cell against its own machine’s limit (the exact optimum’s reads on the same trace, the machine’s highest probed host rate, the card’s datasheet rate). *Bound*: what binds the limit; *host* when the optimum’s reads alone take longer than the GPU’s whole read, so the GPU has slack, *both* when the limit runs experts on the CPU until the two paths take equally long, which is where the all-in-VRAM speed can be exceeded. Ours and FreeToken are launch 2 (FreeToken’s backend carried over from host B’s selection); the 40 and 43.75% budgets do not fit a 24 GB card. [†]On the RTX 3090 FreeToken’s carried-over hybrid backend ran below llama.cpp; its own calibration recommends its offload backend there, which we did not run, so its speed is reported and no ratio against it is printed. Mixtral-8x7B (26 GB of experts, top-2 of 8) is ours against llama.cpp only, on the RTX 3090, launch 1.

Model	Experts on GPU	llama.cpp (tok/s)	FreeToken (tok/s)	Ours (tok/s)	Ours ÷ FreeToken (95% CI)	Ours ÷ llama.cpp	Limit (tok/s)	Bound	Ours, % of limit
RTX 5090, 9950X3D (host B): B_{host} 88 GB/s, B_{gpu} 1792 GB/s; tables 0,0,1,1,2 / 0,0,1,1,2,3,3,4,5									
gpt-oss-120b	11%	34.9	54.0	69.9	1.29 [1.28, 1.31]	2.00	172	host	41
gpt-oss-120b	25%	40.0	85.6	109.2	1.27 [1.25, 1.29]	2.73	430	host	25
gpt-oss-120b	40%	47.7	132.2	152.5	1.15 [1.13, 1.17]	3.20	519	both	29
Qwen3-30B-A3B	12.5%	19.5	38.7	40.0	1.03 [1.02, 1.04]	2.05	96	host	42
Qwen3-30B-A3B	25%	22.1	54.7	63.1	1.15 [1.13, 1.17]	2.86	214	host	30
Qwen3-30B-A3B	43.75%	28.4	103.1	108.2	1.05 [1.04, 1.06]	3.81	308	both	35
RTX 5090, 9950X (host S): B_{host} 71 GB/s, B_{gpu} 1792 GB/s; tables 0,0,1,2,3 / 0,0,1,2,2,3,4,5,6									
gpt-oss-120b	11%	28.9	48.0	57.9	1.21 [1.19, 1.22]	2.00	140	host	41
gpt-oss-120b	25%	33.6	78.7	94.1	1.20 [1.16, 1.22]	2.80	351	host	27
gpt-oss-120b	40%	40.1	122.2	133.6	1.09 [1.08, 1.11]	3.34	514	both	26
Qwen3-30B-A3B	12.5%	16.1	32.7	33.5	1.03 [1.02, 1.04]	2.08	78	host	43
Qwen3-30B-A3B	25%	18.5	51.4	53.9	1.05 [1.04, 1.06]	2.91	174	host	31
Qwen3-30B-A3B	43.75%	23.8	98.3	95.8	0.97 [0.96, 0.99]	4.02	305	both	31
RTX 4090, i5-12400 (job 091): B_{host} 37 GB/s, B_{gpu} 1008 GB/s; tables 0,0,1,2,2 / 0,0,1,1,2,3,4,4,5									
gpt-oss-120b	11%	18.1	23.8	32.7	1.37 [1.35, 1.39]	1.81	74	host	44
gpt-oss-120b	25%	21.0	42.5	55.0	1.30 [1.26, 1.33]	2.62	184	host	30
Qwen3-30B-A3B	12.5%	10.5	18.2	18.8	1.03 [1.02, 1.05]	1.78	41	host	46
Qwen3-30B-A3B	25%	12.3	30.0	31.3	1.05 [1.03, 1.06]	2.54	91	host	34
RTX 3090, i9-11900KF (job 092): B_{host} 45 GB/s, B_{gpu} 936 GB/s; tables 0,0,0,0,0 / 0,0,0,0,0,0,0,0									
gpt-oss-120b	11%	22.8	8.4 [†]	37.3	–	1.64	88	host	42
Qwen3-30B-A3B	12.5%	12.3	9.0 [†]	19.2	–	1.56	49	host	39
Qwen3-30B-A3B	25%	13.8	11.2 [†]	30.1	–	2.18	110	host	27
Mixtral-8x7B Q4_K_M $C=2$ (25%)		8.22	–	8.34	–	1.015 [1.008, 1.024]	–	–	–

53 GB/s by the CPU at the helper count, 43 over the link, 60 with both and 61.5 at best; the model’s FETCH tables 0,0,1,2,2 for gpt-oss and 0,0,1,1,2,3,4,4,5 for Qwen3), and Table 10 the milliseconds per token of the engine states the three jobs realised beside each host’s limit and the model’s values. Table 11 lists the first job (093), the hit-optimal oracle with a window sweep on host O1 (a Ryzen 9 9950X: 44, 46, 49 and 50.5 GB/s; tables 0,0,1,2,3 and 0,0,1,2,3,4,5,6,7), and Table 12 the second (094), MIN with bypass with background admissions beside the hit-optimal oracle on host O2 (a Ryzen 9 7950X: 44, 46, 50 and 50.6 GB/s; the same tables). On O1 and O2 every cell but gpt-oss 40% and Qwen3 43.75% is host-bound at the limit; on O3, whose memory is stronger, the same four.

The single-read oracles. In the fetch oracle the step’s plan is computed before the layer runs from the trace’s record of the current step: the misses MIN with bypass would admit are fetched by the engine’s FETCH operation into their victims’ slots and run from there, and every other miss runs on the CPU; the oracle refuses an admission when no resident’s next use is further than the candidate’s. The single-read prefetch admits, each step, the non-resident experts whose first use is at least d steps away ($d=2$ unpaced, 3 paced), provided the victim’s next use is further than the candidate’s use after that first one; an expert used sooner than d steps away misses on the CPU, or is fetched when the two are combined. The same policies simulated on the trace with the engine’s semantics (the lookahead within the sequence, residents carried over with their next use recomputed at each sequence start) predict the engine’s reads and hit rates: the fetch oracle’s hit rate within 0.4 points and its reads within 0 to 4%, the combined oracle’s hits within two points (Table 9). The simulator the job’s predictions were registered against gave a resident carried over from the previous sequence no next use, so it was evicted first; that over-counted the fetch oracle’s reads by 17% at the two GPU-bound cells, which is why two of its reads clauses failed (Appendix B). Each configuration of **llama-ec-bench** starts with a cold cache in a fresh context. The paced copy path copies one 16 MB piece at a time on a single copier

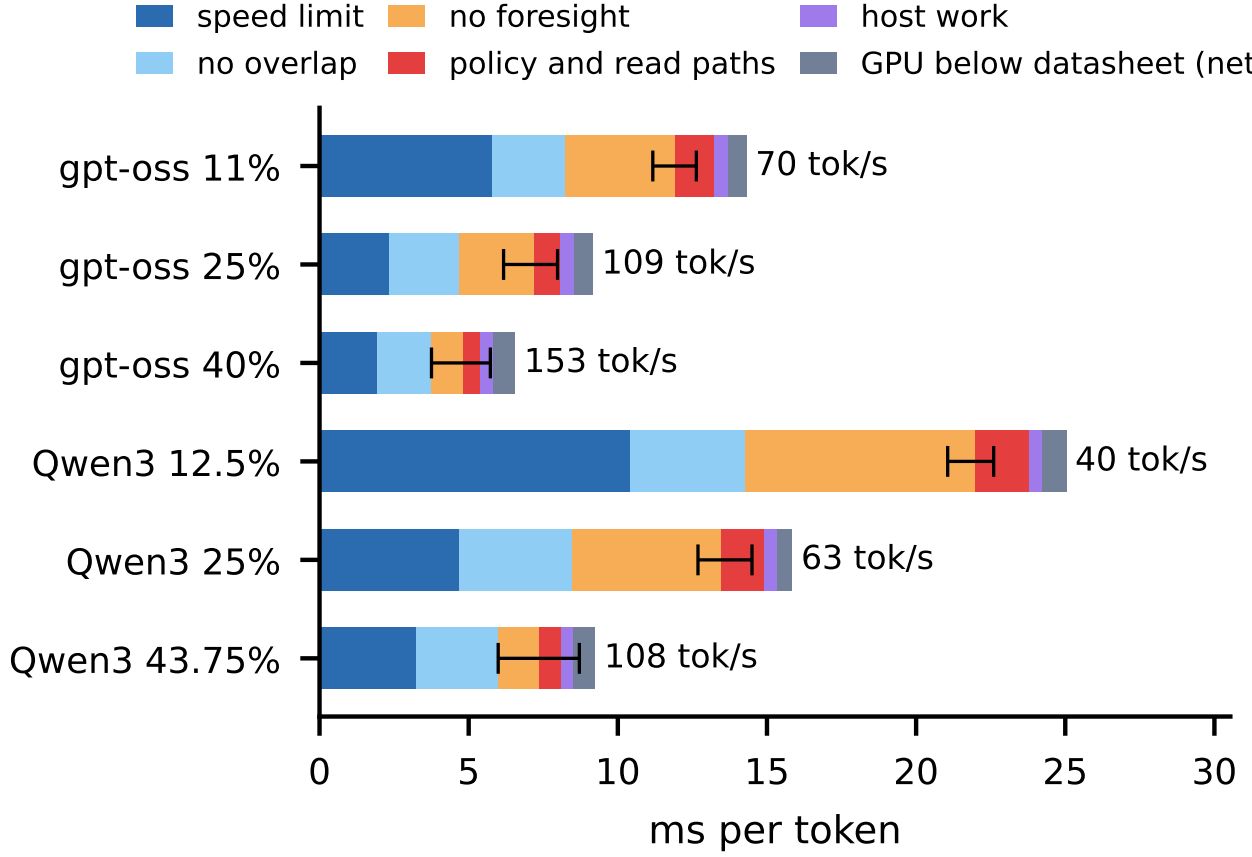


Figure 6: The accounting as a model: Shapley attribution of the gap between our cache’s measured time per token and the bound, for every cell of Table 2 on host B, averaged over all 120 orders in which the five fixes can be applied. Fixes: overlapping host reads with GPU work, which in Eq. (1)’s form also moves experts onto the CPU until the two paths balance; the optimum’s reads instead of the best online policy’s (foresight); the best online policy’s reads at the probe’s highest rate instead of the engine’s reads through Eq. (2) (policy and read paths); the engine’s per-token host work; and GPU kernels at the datasheet rate, the term that closes the model on the measured time. Whiskers: the range of foresight’s marginal cost over the orders.

thread and publishes a landed copy at the start of the next step; the unpaced path copies each admission as one transfer. The hit-optimal oracle’s background admissions drive host memory at 71–82% of the probe’s best rate at the host-bound cells, the prefetching single-read oracles at 64–85% (the paced one 71–82%), the online policy itself at 66–82%, the fetch oracle at 42–67%. The predictions and their outcomes are in `prereg/foresight_outcome_095.md`: of 56 clauses, 44 held and 12 failed, all of size (Appendix B).

The double-reading oracles. Both read the routing of the next W decode steps; their copies ride the paced admission path, and an expert still in flight at its step runs on the CPU like any other miss. The *hit-optimal* oracle admits the experts those steps will route to, soonest first, evicting the resident whose next use is furthest (Belady within the window), for one future use as readily as for ten. The *bytes-optimal* oracle is MIN with bypass with its admissions made through the background path: a miss is served where it is, by the CPU or a demand copy, and admitted only if its next use comes before the furthest next use among the residents. With the whole sequence in view the hit-optimal oracle runs $1.40\times [1.38, 1.41]$ the online policy at gpt-oss 25% and $1.25\times [1.24, 1.26]$ at Qwen3 25% on host O1, $1.63\times$ and $1.41\times$ on O2, and $1.48\text{--}1.62\times$ at the two GPU-bound cells; half of its gain at gpt-oss 25% arrives between $W=4$ and 16 (9.2 tokens by log interpolation; Qwen3 25%: 6.1 against the trace’s 3.9). Where the host binds hardest it loses, 3–8% at gpt-oss 11% and Qwen3 12.5% on both hosts, and at every window on O1 ($0.78\text{--}0.92\times$):

Table 9: The single-read oracles (job 095; RTX 5090, Ryzen 9 9950X, host memory 61.5 GB/s at best) beside the hit-optimal one. Per cell: the online policy’s speed and its share of this host’s limit; for each oracle with the whole sequence in view, its ratio to the online policy (paired, 95% interval), hit rate, host reads per token (CPU misses + fetches + admissions) as a multiple of the optimum’s (each read counted once), share of the limit and the calibrated model on its own counters against its measured time. *fetch*: MIN with bypass, admitted misses fetched into their slot this step; *lead*: the scheduled single-read prefetch (first use at least two steps away); *both*: fetch and lead; *paced*: both on the paced copy path with a three-step lead; *hit-opt.*: the hit-optimal oracle (Belady prefetch without bypass, unpaced). The simulation columns give the trace’s prediction of reads and hits for fetch and both.

Cell	online			fetch				both				hit-opt.				lead		paced		simulated						
	tok/s	% lim.	ratio	hits	reads/R*	% lim.	model	ratio	hits	reads/R*	% lim.	ratio	hits	reads/R*	% lim.	ratio	% lim.	ratio	% lim.	fetch	reads	hits	both	reads	hits	
gpt-oss 11%	55.5	46	1.29	[1.28, 1.29]	72	1.04	59	-8%	1.39	[1.39, 1.40]	86	1.23	64	0.91	[0.90, 0.92]	98	1.77	42	1.26	58	1.45	67	39	73	46	87
gpt-oss 25%	90.3	30	1.32	[1.31, 1.33]	89	1.08	39	-13%	1.49	[1.48, 1.50]	97	1.45	44	1.14	[1.12, 1.15]	99	2.11	34	1.45	43	1.66	50	16	89	22	99
gpt-oss 40%	128.8	25	1.25	[1.24, 1.27]	95	1.14	32	-14%	1.39	[1.37, 1.41]	99	1.60	35	1.27	[1.25, 1.28]	99	2.20	32	1.38	35	1.49	38	7	95	12	100
Qwen3 12.5%	32.3	48	1.34	[1.34, 1.35]	74	1.04	64	-9%	1.42	[1.41, 1.43]	84	1.20	68	0.69	[0.68, 0.71]	77	2.47	33	1.22	58	1.41	68	100	74	115	84
Qwen3 25%	51.8	34	1.38	[1.37, 1.39]	88	1.04	48	-15%	1.48	[1.46, 1.49]	96	1.42	51	1.06	[1.05, 1.08]	99	2.00	37	1.41	48	1.67	57	45	88	61	97
Qwen3 43.75%	91.4	30	1.26	[1.25, 1.28]	96	1.08	38	-17%	1.41	[1.39, 1.42]	99	1.62	42	1.27	[1.26, 1.29]	100	2.29	38	1.40	42	1.50	45	15	96	23	100

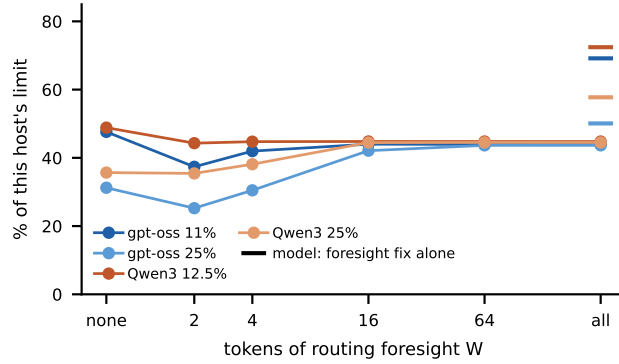


Figure 7: The first oracle job (093, host O1): speed as a percentage of this host’s limit against the window W of future routing the hit-optimal oracle sees, at the four host-bound cells. “none” is the online policy; “all” the whole sequence; points are connected for the trend only. The tick at the right is the model’s foresight-only term for the cell, the speed it predicts with that one fix; the oracle’s host traffic sits at 89–94% of the probe’s best rate at every window, so its speed is R^* over its reads times that utilisation, 44–45% of the limit with the whole sequence in view.

it reads more host bytes than the online policy (82 and 195 experts per token against 63 and 160), $2.02\text{--}2.18\times$ the optimum’s R^* , because Belady admits an expert used once where the optimum bypasses it and re-copies what it evicts (its admissions, each one read, are most of its reads; a miss served on the CPU before its copy lands is the second read of that expert). An unpaced copy path, run at the four host-bound cells of O1, reaches 98–99% hits at three of them and still loses at gpt-oss 11% and Qwen3 12.5% (where it hits 75%), so copies that arrive in time do not remove the loss there: the extra bytes cause it. The bytes-optimal oracle gains 13–23% at the four cells of 25% and above and 0–3% at the two below (6% with a 16-token window); its misses fall (30 to 20 and 85 to 64 per token at 25%) but its total reads, misses plus the admissions that read each admitted expert again, are $1.73\text{--}2.04\times R^*$ and exceed the online policy’s by 4–5% at the two lowest budgets; the model on its own counters predicts its time within 9% at every cell, and the bytes it saves account for 4–21 points of its 13–23% gain at the four cells (most of it at gpt-oss 40% and Qwen3 43.75%, a third at Qwen3 25%). The hit-optimal oracle reads $1.01\text{--}1.13\times$ the bytes-optimal one’s host bytes on O2 and gains 41–63% at the same four cells, and the model, with its GPU term calibrated on the online run, over-predicts those runs’ time per token by 37–53% (on O1, 24–39%): its gain is the overlap of copies issued ahead of their use with the step, which the model charges in sequence. Pacing the online policy’s own copies changes it by under 1% ($0.997\text{--}1.005\times$). With no FETCH table (every miss on the CPU) the online policy runs 8–15% slower at the four host-bound cells; with the engine’s within-layer overlap of a layer’s CPU misses with its GPU hits turned off it changes by under 1% at every cell (0.5% faster to 0.6% slower), so the overlap the accounting charges is across the step and in rebalancing, not within a layer. The bytes-optimal oracle with a 16-token window keeps 34–176% of the whole-sequence gain at the five cells where that gain’s interval excludes zero (more than all of it at gpt-oss 11%, where the shorter window admits less).

Table 10: The accounting, measured: milliseconds per token of the engine’s states that the oracle jobs realised, on each oracle host, beside that host’s limit and the model’s values for the states it prices; O1–O3 ran jobs 093–095 in a fixed order, O4 and O5 job 096 in a shuffled order. *online*: the deployed policy; *foa*: the same with each admission fetched into its slot in the step that misses it (one read, no foresight); *no ovl.*: its within-layer overlap off; *all CPU*: no FETCH table; *bypass*: MIN with bypass with background admissions (two reads per admitted expert); *hit-opt.*: Belady prefetch without bypass, which admits what the optimum bypasses, unpaced, and the same on the paced copy path (the first *paced* column; O1 and O2 ran only that form); *nb2*: Belady within a two-step lead with each admission read once; *fetch*: MIN with bypass, admitted misses fetched into their slot (one read, serialised: the model’s foresight-only state $v(F)$); *both*: fetch and the scheduled single-read prefetch (one read, overlapped: the model’s $v(O, F)$); the second *paced*: both on the paced copy path with a three-step lead; *limit*: the limit on that host; $v(F)$, $v(O, F)$, $v(\text{all})$: the model.

Host	Cell	online	foa	no ovl.	all CPU	bypass	hit-opt.	paced	nb2	fetch	both	paced	limit	$v(F)$	$v(O, F)$	$v(\text{all})$
O1	gpt-oss 11%	21.12	–	21.01	22.91	–	–	22.88	–	–	–	–	10.06	14.54	11.27	10.06
O1	gpt-oss 25%	12.89	–	12.91	14.21	–	–	9.22	–	–	–	–	4.03	8.04	5.10	4.03
O1	gpt-oss 40%	8.90	–	8.95	–	–	–	5.99	–	–	–	–	1.97	5.75	3.84	1.97
O1	Qwen3 12.5%	36.92	–	36.91	43.29	–	–	40.29	–	–	–	–	18.04	24.90	19.29	18.04
O1	Qwen3 25%	22.73	–	22.76	26.60	–	–	18.22	–	–	–	–	8.11	14.04	9.22	8.11
O1	Qwen3 43.75%	12.71	–	12.67	–	–	–	8.12	–	–	–	–	3.32	8.25	5.51	3.32
O2	gpt-oss 11%	20.85	–	–	–	20.17	–	21.57	–	–	–	–	10.04	14.27	11.96	10.04
O2	gpt-oss 25%	12.72	–	–	–	10.77	–	7.80	–	–	–	–	4.02	7.86	5.86	4.02
O2	gpt-oss 40%	8.79	–	–	–	7.13	–	5.62	–	–	–	–	1.97	5.63	3.78	1.97
O2	Qwen3 12.5%	36.64	–	–	–	36.58	–	38.74	–	–	–	–	18.00	24.70	20.06	18.00
O2	Qwen3 25%	22.24	–	–	–	19.67	–	15.80	–	–	–	–	8.09	13.58	9.90	8.09
O2	Qwen3 43.75%	12.02	–	–	–	10.11	–	7.42	–	–	–	–	3.32	7.56	4.90	3.32
O3	gpt-oss 11%	18.01	–	–	–	–	19.79	–	–	14.00	12.91	12.39	8.26	12.63	9.49	8.26
O3	gpt-oss 25%	11.08	–	–	–	–	9.74	–	–	8.42	7.44	6.66	3.31	7.05	4.28	3.31
O3	gpt-oss 40%	7.77	–	–	–	–	6.14	–	–	6.20	5.58	5.20	1.95	5.24	3.56	1.95
O3	Qwen3 12.5%	30.95	–	–	–	–	44.54	–	–	23.02	21.78	21.89	14.81	20.35	16.06	14.81
O3	Qwen3 25%	19.32	–	–	–	–	18.17	–	–	14.01	13.09	11.58	6.66	11.68	7.76	6.66
O3	Qwen3 43.75%	10.94	–	–	–	–	8.59	–	–	8.66	7.78	7.28	3.30	7.24	4.76	3.30
O4	gpt-oss 11%	20.59	20.18	–	–	20.11	19.78	22.03	19.64	15.13	14.06	13.78	10.02	13.77	11.63	10.02
O4	gpt-oss 25%	12.43	12.07	–	–	10.55	9.76	8.13	9.61	8.72	7.65	6.87	4.01	7.40	5.34	4.01
O4	gpt-oss 40%	8.39	8.19	–	–	6.91	6.15	5.38	6.03	6.28	5.59	5.21	1.97	5.12	3.20	1.97
O4	Qwen3 12.5%	36.36	35.83	–	–	36.54	48.80	39.66	31.64	25.29	24.01	24.58	17.97	23.98	19.88	17.97
O4	Qwen3 25%	22.09	21.75	–	–	20.06	17.74	16.93	17.57	14.60	13.50	12.24	8.08	13.12	9.54	8.08
O4	Qwen3 43.75%	11.94	11.74	–	–	10.09	8.44	7.17	8.33	8.69	7.72	7.29	3.32	7.32	4.56	3.32
O5	gpt-oss 11%	21.26	21.31	–	–	21.47	33.34	25.57	33.03	18.34	17.97	16.39	9.94	15.22	11.20	9.94
O5	gpt-oss 25%	13.01	13.35	–	–	11.18	15.87	11.17	15.60	10.60	10.54	8.68	3.98	9.13	5.06	3.98
O5	gpt-oss 40%	8.73	9.22	–	–	7.16	8.34	6.25	8.13	7.27	6.87	5.91	1.97	6.26	3.62	1.97
O5	Qwen3 12.5%	37.63	37.42	–	–	40.11	69.42	44.89	49.11	31.37	31.27	30.01	17.82	27.95	19.27	17.82
O5	Qwen3 25%	23.24	23.27	–	–	21.12	30.78	21.76	30.47	18.51	19.64	16.03	8.01	16.77	9.15	8.01
O5	Qwen3 43.75%	12.49	12.92	–	–	10.32	12.08	9.00	11.86	10.21	9.61	8.23	3.31	8.94	5.02	3.31

Why the bytes-optimal oracle’s misses stay above the optimum’s reads. Besides the second read, its misses alone are $1.31\text{--}1.47\times$ the optimum’s reads at the four host-bound cells (at the two GPU-bound ones the ideal simulation below is itself 6–10% above the optimum, which sees across sequences). The engine’s paced copy path moves one 16 MB piece at a time from pageable host memory on a single copier thread, behind the demand copies, publishes a landed copy at the start of the next step, and admits nothing while 64 copies are queued; at the 16–30 admissions per token (210–285 MB) of the two lowest budgets it lags the admissions by steps, and an expert still in flight is a miss the CPU reads again. The last four columns of Table 12 simulate MIN with bypass on the measured routing trace with the engine’s own semantics (the lookahead within the sequence, residents carried over, no queue cap) and a copy that lands d steps after it is issued, during which the expert misses and is not re-admitted: at $d=1$, the ideal, the misses come within 0–4% of the limit’s optimum at the host-bound cells; the engine’s misses meet the simulation at $d=2.8\text{--}4.7$ there and at 3.0–3.5 at the GPU-bound cells, three to five steps. At the fitted latency the simulation admits $1.1\text{--}1.9\times$ what the engine admitted, so the queue cap, not only the latency, keeps the engine’s misses above the ideal. The predictions and their outcomes are in `prereg/foresight_outcome.093.md` and `094.md`: of 36 clauses for the first job, 9 held, 2 held on the point estimate, 21 failed and 4 were untested; of 38 for the second, 12 held, 1 on the point estimate, 24 failed and 1 was untested (Appendix B). The second job’s failures are of size, not direction: the bypass oracle’s misses were predicted within 15% of the optimum’s reads and came 29–47% above (the copy latency and the queue cap); its hit rate was predicted within 4 points of the optimum’s and fell 5–11 points short at three cells; its gain was predicted at 10–45% at the host-bound cells and came below that at the two lowest budgets; the model was predicted to over-predict it by 10–40% at the host-bound cells and over-predicted by 5–9%; and the two oracles were predicted within 15% of each other at 25% and above, where the hit-optimal one was $1.24\text{--}1.38\times$ faster.

Table 11: The hit-optimal oracle in the engine (job 093; RTX 5090, Ryzen 9 9950X, host memory 50.5 GB/s at best). tok/s of the online policy and of the oracle with the whole sequence in view; their ratio, paired by sequence, with its 95% interval; hit rate and misses per token of each (experts read from host memory by the CPU or by a demand copy), the optimum’s reads in parentheses, and the oracle’s background admissions (Belady prefetch without bypass: it admits what the optimum bypasses); speed as a percentage of this host’s limit; the accounting’s foresight-only term $v(F)$ for the cell in tok/s; the share of that term the oracle recovered and the share of the gap to the limit it closed.

Cell	online	oracle	ratio	hits % (opt.)	misses/token (opt.)	oracle adm.	% of limit	$v(F)$	recovered %	closed %
gpt-oss 11%	47.4	43.7	0.92 [0.91, 0.93]	58 → 67 (73)	60 → 47 (38)	35	47.6 → 44.0	68.8	−27	−16
gpt-oss 25%	77.6	108.5	1.40 [1.38, 1.41]	79 → 92 (89)	30 → 11 (15)	20	31.2 → 43.7	124.4	76	41
gpt-oss 40%	112.4	166.9	1.48 [1.46, 1.51]	89 → 98 (95)	15 → 3 (7)	12	22.1 → 32.8	174.0	92	42
Qwen3 12.5%	27.1	24.8	0.92 [0.91, 0.92]	60 → 66 (75)	155 → 132 (97)	63	48.9 → 44.8	40.2	−28	−18
Qwen3 25%	44.0	54.9	1.25 [1.24, 1.26]	78 → 89 (89)	85 → 42 (43)	46	35.7 → 44.5	71.2	52	31
Qwen3 43.75%	78.7	123.1	1.56 [1.53, 1.59]	92 → 99 (96)	32 → 5 (14)	25	26.1 → 40.8	121.3	103	49

Table 12: The bytes-optimal oracle (MIN with bypass) beside the hit-optimal one on the second oracle host (RTX 5090, Ryzen 9 7950X, host memory 50.6 GB/s at best). Per cell: the online policy’s speed; each oracle’s ratio to it with the whole sequence in view (paired, 95% interval), its hit rate, its misses per token (experts read from host memory by the CPU or by a demand copy; its background admissions in parentheses: the bypass oracle’s are this step’s CPU-served misses, each read a second time by the copy, the hit-optimal one’s are prefetches of what the optimum would bypass as readily as of what it would admit) and the calibrated model on its own counters against its measured time; the optimum’s reads and hit rate from the trace; and MIN with bypass simulated on the trace with the engine’s lookahead (within the sequence) under a copy latency of 1 (ideal), 2 and 3 steps, with the latency at which the simulation meets the engine’s misses.

Cell	online	bypass oracle, $W=all$					hit-optimal oracle, $W=all$					optimum		simulated misses/token				
		ratio	hits	misses (adm.)	model	ratio	hits	misses (adm.)	model	reads	hits	$d=1$	2	3	fit			
gpt-oss 11%	48.0	1.03	[1.03, 1.04]	65	50 (16)	+7%	0.97	[0.96, 0.98]	71	42 (39)	+19%	38.3	73	39	43	47	3.9	
gpt-oss 25%	78.6	1.18	[1.18, 1.19]	86	20 (9)	+6%	1.63	[1.61, 1.66]	95	7 (23)	+53%	15.3	89	16	17	19	4.7	
gpt-oss 40%	113.8	1.23	[1.22, 1.24]	94	9 (4)	+2%	1.56	[1.54, 1.59]	98	3 (12)	+39%	6.8	95	7	8	9	3.5	
Qwen3 12.5%	27.3	1.00	[1.00, 1.01]	64	138 (30)	+5%	0.95	[0.94, 0.95]	67	128 (65)	+11%	96.5	75	97	122	143	2.8	
Qwen3 25%	45.0	1.13	[1.12, 1.14]	83	64 (21)	+9%	1.41	[1.39, 1.43]	91	34 (52)	+37%	43.4	89	44	54	61	3.4	
Qwen3 43.75%	83.2	1.19	[1.18, 1.20]	95	19 (9)	+1%	1.62	[1.58, 1.65]	99	3 (27)	+47%	13.7	96	14	17	19	3.0	

Where a token’s time goes at a GPU-bound budget. On Qwen3 at 43.75% Nsight Systems splits a token of our cache (FreeToken in parentheses) into 4.93 (5.20) ms of expert copies over PCIe, 2.29 (2.21) of resident-expert products, 1.65 (1.34) of attention projections, 0.49 (0.37) of attention, 0.38 of output head and 0.97 (0.40) of norms, small kernels and cache control, with neither system overlapping its copies with GPU compute. Turning off the within-layer overlap of a layer’s CPU misses with its GPU hits changes speed by under 1% at every budget, and predicting the next layer’s experts [23, 65] never lowers host bytes.

G The trace study in detail

Policies. Table 13 scores LRU, LFU, decayed frequency, ARC, S3-FIFO and a profiled static cache against MIN and against Belady within a window of W future tokens, at three budgets per model, on each model’s own sampled text (9 traces, the cache carried across conversations). Without admitting every miss, the online policies are close to one another and far from MIN: the best reads 35–110% more, decayed frequency is best at 12 of 27 budget points and S3-FIFO at 10. Where the cache turns over every token ($C/k \leq 1.5$) the six are within 4% of one another; from $C/k=2$ LRU reads 2–11% more than decayed frequency on eight of nine models, and classic LFU fails at large budgets. Adding *admit every miss* (every miss copied into the slot of the lowest-scored resident not requested now) changes the ranking: it is best at 18 of 27 points, ties at 5, and loses by at most 2.7%; it saves up to 5.8% over the previous best, and the best online policy still reads 35–110% more than MIN (`prereg/policy_aa.json`). At the small budgets almost every resident turns over within a few tokens, and the expert just requested is a good bet. In our engine admitting every miss reads all of its misses over the link, so its value in time depends on the link (Section 5). A static cache profiled on other text reads 1.4–13.2× MIN because the hot set moves: the C most requested experts of a layer change by 22–58% between windows of 100 tokens and 33–75% between windows of 1,000.

The horizon, before and after the stronger baseline. Against decayed frequency with each admission fetched once (the study’s original baseline), MIN reads 26–56% fewer experts (median 41%), and the window that closes half

Table 13: Host reads per token relative to the exact optimum (Belady’s MIN with bypass over the whole future), median over nine models at three budgets, and the range over all 27 model–budget cells. Event-atomic replay of each model’s own sampled text; every online policy is scored on the last 90% of the trace. Mixtral (8 experts) uses $C = 2, 3$ and 4.

Policy	$C=E/8$	$E/4$	$3E/8$	range
LRU	1.44	1.77	2.03	1.37–2.26
LFU	1.44	1.93	2.40	1.37–6.15
Decayed frequency	1.45	1.70	1.94	1.35–2.10
with hysteresis κ	1.46	1.75	2.00	1.36–2.55
ARC	1.44	1.76	1.99	1.37–2.23
S3-FIFO	1.44	1.69	1.94	1.38–2.11
Static, profiled	1.92	2.74	3.48	1.43–13.24
Belady, $W=1$	1.18	1.45	1.69	1.07–2.06
Belady, $W=4$	1.01	1.13	1.31	1.00–1.95
Belady, $W=16$	1.00	1.00	1.01	1.00–1.66

of the gap fitted $W_{50} \approx 0.59 (C/k)^{1.33}$; admitting every miss lowers the baseline at 23 of 26 points by up to 6.7%, raises W_{50} by a median 9% (at most 42%), and refits it to $0.67 (C/k)^{1.31}$, with a leave-one-model-out exponent range of 1.25–1.36 and, without the points below one token, 1.35 (it was 1.51 against the original baseline). Ninety percent of the gap needs a median of $2.7 C/k$ tokens.

A learned admission order. We trained a logistic model of whether an expert is requested within the next C/k tokens on features the engine has when it plans a step (decayed request counts at five half-lives, the time since the last request, whether the expert is requested now, and transition scores from the previous token’s experts at the same layer and, through an 8-bit table, at every layer), fitted on each model’s mixed-domain trace and replayed on the AIME routing, so no test text is seen in training (Table 14). With each admission read once it reads 6–12% fewer experts than decayed frequency with the same single-read admission at the four host-bound budgets, closing 17–30% of that policy’s excess over MIN; admitting every miss alone closes most of that at Qwen3 12.5% and little at gpt-oss (prereg/learned.offline.json and the research checks). In the engine (job 097, predictions committed before launch) it reads 6–13% fewer experts than the single-read online policy at the host-bound budgets on both hosts, but runs $1.01\text{--}1.03\times$ it on O4’s machine and $0.90\text{--}0.94\times$ on host O6: the reads it saves are CPU reads, bought with more fetches in the step, each a copy its layer waits on, and with $237\text{--}585\ \mu\text{s}$ of host time per token (over all six budgets).

Speculation is not foresight. Speculative decoding verifies K draft tokens in one forward pass, so a layer’s experts for K positions are requested together and a missed expert is read once for all of them. On the same traces, with drafts accepted at rate α until the first rejection, batched verification saves an online cache a median 15–32% of its reads at $K=8$ when rejected drafts cost nothing, the equivalent of a window of two to four tokens; once the rejected drafts’ experts are counted, the saving is 8–12% at $\alpha=0.9$ and becomes a loss of 18–20% at $\alpha=0.8$, because the union of K positions’ expert sets is 20–79% of Kk . MIN itself gains little from batching (–7 to 15%). Systems that use drafts as a look-ahead sensor [34] compare against a baseline that already pays for verification; ours charges the rejected drafts to a cache that does not speculate.

Table 14: A learned admission order, offline: host reads per token on the AIME-25 routing the engine runs, of the decayed frequency with each admission read once (single read; within 10% of the engine’s fetch-on-admit reads on hosts O4 and O5), the learned order with single-read admission (trained on the model’s mixed-domain trace; 8-bit cross-layer table, as in the engine), Belady within one and four tokens of perfect foresight, and MIN with bypass in its no-evict form (Table 16). *closed*: the share of the single-read policy’s excess over MIN that the learned order removes. [†]host-bound.

cell	single read	learned	W=1	W=4	MIN	closed
gpt-oss 11% [†]	59.4	56.0	54.4	47.3	39.0	17%
gpt-oss 25% [†]	28.3	26.1	27.3	25.1	15.5	17%
gpt-oss 40%	14.3	12.6	14.0	13.2	6.8	23%
Qwen3 12.5% [†]	166.2	146.3	134.1	109.1	100.4	30%
Qwen3 25% [†]	84.2	74.8	74.1	62.8	44.3	24%
Qwen3 43.75%	28.8	26.3	27.7	25.3	13.8	17%

Table 15: Partial foresight on the AIME routing the engine runs: host reads per token of admitting every miss and of MIN with one read, and the share of the gap between them that each window closes (Table 1). Recall 0.5: each future expert kept with probability 0.5 and replaced by a random one (fill) or left out (drop); means of three seeds. Linear: the ridge forecaster of Appendix H with a 4-token window. $\kappa=1$: the same exact 4-token window with the deployed margin instead of admitting every miss beyond it, as a share of its own gap.

Budget	Reads per token		Exact window W (tokens)					Recall 0.5, fill / drop		Linear	$\kappa=1$
	every miss	MIN	1	2	4	8	16	$W=8$	all	$W=4$	$W=4$
gpt-oss 11%	58.6	39.3	0.18	0.31	0.52	0.81	0.98	0.29 / 0.29	0.43 / 0.48	0.13	0.62
gpt-oss 25%	28.2	16.0	0.07	0.14	0.25	0.42	0.66	0.14 / 0.13	0.46 / 0.47	0.05	0.34
Qwen3 12.5%	150.0	100.3	0.31	0.51	0.77	0.96	0.99	0.42 / 0.43	0.48 / 0.52	0.19	0.88
Qwen3 25%	79.6	44.9	0.16	0.28	0.46	0.70	0.93	0.29 / 0.28	0.49 / 0.53	0.11	0.60

H Partial foresight in detail

The window policies. At step g the engine plans the step’s misses before it runs, from the trace’s record of step g (the routers give it layer by layer in a deployable system) and from a view of records $g+1..g+W$ of the same problem. A miss is processed in request order: it takes a free slot; otherwise the victim is a resident not requested now that is not seen in the window (lowest decayed score), or, if every resident is seen, the one seen furthest; a miss seen in the window is admitted unless the victim is seen sooner, and a miss not seen is admitted if the victim is not seen either (admit every miss beyond the window; with the deployed margin $\kappa=1$, only if its score beats the victim’s by one). On the panel admissions are fetched in the step, so every miss is read once; on the deployed path of the follow-up the host’s fetch table splits the misses between the CPU and an in-step copy, and an admission served by the CPU is copied afterwards, a second read. In a degraded view each expert of a future record is kept with probability r , decided by a hash of (seed, g , record, layer, position), so step g sees one view throughout and step $g+1$ an independent one; a dropped expert is replaced by a uniformly drawn one (fill: precision and recall r) or left out (drop: recall r , precision 1). `scripts/value_map.py` replays the plan bit for bit: on a CPU build the engine’s reads equal the replay’s in 30 configurations at three budgets (`cputest099`), and on the panel’s GPUs, where the routing in the run can differ from the recorded trace by numerics, within 1.7% at both gpt-oss budgets. The panel ran the first 20 of the 30 problems; the table’s reads are on all 30.

What the table says. Horizon and accuracy trade: at gpt-oss 11% an 8-token window at recall 0.5 closes 0.29 of the gap, about what two exact tokens close (0.31), and the whole future at recall 0.5 closes 0.43. Fill and drop cost about the same for windows of up to 16 tokens; with the whole future in view at low recall, dropping costs less (the largest difference, 0.12, is there). A larger cache needs a longer window: at 25% the same exact 4-token window closes 0.25 of the gap, against 0.52 at 11%.

A realisable forecaster. Ridge regression from the all-layer routing of step t to that of step $t+h$ ($h=1..8$), fitted on each model’s mixed-domain own text and applied to the AIME routing, forecasts the top k experts of each layer with a precision of 0.48 one token ahead and 0.26 four ahead on gpt-oss (0.54 and 0.32 on Qwen3); a decayed request count, the online policy’s own signal, scores 0.30 and 0.28. Driving the window policy, the ridge forecasts close 3–15% of the gap on gpt-oss and 6–19% on Qwen3 over windows of 1–8 tokens (`prereg/forecaster_linear.json`). Its precision

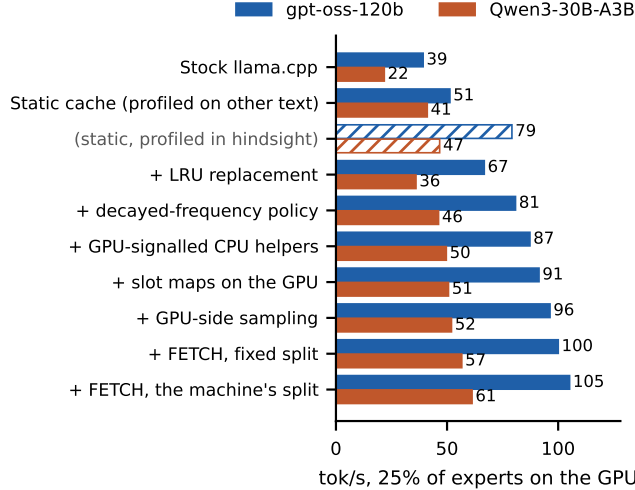


Figure 8: Ablation at 25% of experts on the GPU (15 AIME-25 problems, 256 tokens, host A). Each step adds one component to the one above; the hatched row is not a step: a static cache of each layer’s 32 most requested experts counted on the measured problems themselves.

falls with the horizon, so over a window it is closer to 0.3 than to its one-token value; at that average precision a window with random errors closes about as much on gpt-oss, and more on Qwen3, where the forecaster’s correct forecasts are mostly experts the cache already holds. A sequence model reports recall above 90% one token ahead with $k+3$ candidates [50]; its precision is then at most $k/(k+3)$, 0.57 for gpt-oss and 0.73 for Qwen3. A recurrent forecaster in that spirit (a GRU of 512 units over the same all-layer vectors, one output head per horizon, trained on the same text; `scripts/forecaster_gru.py`) reaches a precision of 0.40 and 0.26 one and four tokens ahead on gpt-oss (0.46 and 0.31 on Qwen3), a recall at $k+3$ of 0.53 and 0.54, and closes 2–11% and 6–17% of the gap: no better than the linear model. Our training text is smaller than SeqMoE’s and differs from the test text, so this bounds what such a model does across text, not what it can do in domain.

I The expert cache in more detail

This section holds the measurements of Section 2 that the main text summarises: where the cache’s gain comes from, the choice of the FETCH split, the hosts with half the link, long outputs, FreeToken’s own headline model, two other llama.cpp-based systems, output parity, and the cost of prompts and batches. Job numbers refer to the scripts on the `gpu` branch.

Where the gain comes from. Figure 8 builds the system up from stock llama.cpp on one machine. Placing each layer’s hot experts is the first gain: a static cache of the 32 experts per layer most requested on other text (the models’ own MATH-500 outputs) runs $1.31\times$ and $1.89\times$ stock, which pins whole layers instead. Its hit rate (52% and 67%) depends on how well the profile matches the workload: profiled in hindsight on the measured problems it hits 75% and 72% and runs faster (hatched). Replacing experts online closes that gap without a profile, if admissions are rationed: LRU admits every miss, and on Qwen3 its copies raise host reads from 127 to 188 experts per token, so it runs $0.88\times$ the static cache ($1.30\times$ on gpt-oss); the decayed-frequency policy reads 41 and 118 per token, runs $1.58\times$ and $1.12\times$ the profiled static cache, and the hindsight cache runs 0.98 [0.95, 1.01] and 1.01 [0.98, 1.04] times as fast as it. On traces LRU misses only 2–11% more than decayed frequency at these budgets (Section 5); the rest of its cost here is the copies it admits. On host S, with everything else as in the full system, LRU runs 6–21% slower than decayed frequency across the six cells of Table 2 and trails FreeToken at four of them: the admission policy is most of the lead over FreeToken (77–84% of it at gpt-oss 25 and 40%, more than the whole of it at gpt-oss 11% and Qwen3 12.5 and 25%). The GPU-signalled CPU helpers, GPU slot maps, GPU sampling and FETCH add 2–9% each, and the model’s FETCH table 5–8% over the fixed one. The full system runs $2.68\times$ (gpt-oss) and $2.81\times$ (Qwen3) stock llama.cpp.

Table 16: The bound under each tightening, tokens per second on host B, and our cache as a percentage of the bound as published (exact optimum, highest probed host rate, datasheet GPU rate) and with every tightening at once. *no-evict*: Belady’s MIN with bypass that keeps a served expert for the step; *exact*: the same, free to evict an expert after serving it; *pool*: one pool of *LC* slots shared across layers; *median*: B_{host} the median of six concurrent probe samples instead of the highest; *GPU*: the GPU term from the measured all-in-VRAM time instead of the datasheet rate; *layers*: the per-layer sum, which no system without cross-layer prefetch can beat; *all*: exact, pool, median and GPU together (a warm start, not included, changes the bound by at most 1.4%).

Model	Experts on GPU	no-evict	exact	pool	median	GPU	layers	all	Ours, % of exact	% of all
gpt-oss-120b	11%	169	172	182	159	172	163	168	41	41
gpt-oss-120b	25%	427	430	474	397	279	365	278	25	39
gpt-oss-120b	40%	519	519	519	517	279	500	278	29	55
Qwen3-30B-A3B	12.5%	92	96	101	89	96	93	93	42	43
Qwen3-30B-A3B	25%	209	214	234	197	192	193	191	30	33
Qwen3-30B-A3B	43.75%	308	308	308	307	192	298	191	35	57

How tight is the bound? Table 16 changes one assumption at a time on host B. Pooled slots need 4.5–18.6% fewer reads and raise the host-bound limits by 5–10%; starting MIN warm changes it by at most 1.4%; the median probe sample instead of the highest lowers the host-bound limits by 8%. The GPU term moves most: at batch size 1 the all-in-VRAM runs reach only 52% (gpt-oss) and 61% (Qwen3) of the datasheet rate, and with the measured rate the limits at 25% and above fall to 279 and 192 tok/s, 8–9% above the all-in-VRAM speed. In the running example (gpt-oss at 25%) the limit on host B is 430 tok/s (279 with the measured GPU rate) and the cache runs 109 tok/s. With every expert in its slots the cache matches stock llama.cpp ($0.997\times$ and $1.047\times$). The bound assumes an expert run on the CPU is read from DRAM (MIN’s shortest re-read distance is 0.5 GB of host traffic) and that computation runs at memory speed. The routing traces are each model’s own greedy text on the 30 prompts; replayed through a simulator of the deployed policy they reproduce the engine’s hit rates within 5.3 points.

Reaching the read time. The microbenchmark of Section 3 (`jobs/ec2/readsched.cu`) takes MIN’s misses per layer and step from `scripts/readsched.py` and runs no model. Usable cores minus two CPU threads read their share of host memory from an ordinary buffer, as the engine’s helpers do, and the copy engine copies the rest, 13.25 MB at a time, from pinned memory into GPU slots; both buffers rotate through 4 GiB so that no cache holds them. *Per layer* splits each layer’s experts, whole, between the two paths at the probe’s rates and waits for both before the next layer; *per token* sums the same split over the token and waits once; *link only* sends every read over PCIe, as admitting every miss does (MIN with one read sends only its admissions); *CPU only* reads everything on the CPU. Table 17 gives each host. Waiting per layer instead of per token costs at most 1 point of the host term on the two AMD hosts and 7 on the Intel host, whose link is the slowest; there, reading everything on the CPU beats the per-layer split. The split ignores which reads are admissions; Section 3 gives the cap that the link puts on MIN’s one-read schedule (`prereg/readsched.json` per host).

Choosing the split per machine. On host B, the model’s table beats the fixed table we used before at every budget of both models, by 3.3–8.0% (launch 1). Table 18 shows why at one budget: a single missed expert should run on the CPU (about $155\mu\text{s}$, hidden behind the GPU’s other experts) rather than be copied (about $225\mu\text{s}$, on the critical path); that entry alone is worth 3.7%. Fetching everything, FreeToken’s offload strategy expressed in our engine, is the worst table, and running everything on the CPU the second worst. The model’s table was within 1% of the best of five alternatives on launch 1 and tied with it on launch 2. Given each run’s measured misses, Eq. (3) ranks the tables correctly except law against lite, which are within 1%, and overstates their differences about $1.8\times$.

Machines with half the link. On RTX 5090s whose links we measured at 27–30 GB/s instead of 53 (jobs 085, 085b and 088: gpt-oss next to a Core i9-14900K, Qwen3 next to a Ryzen 9 7900 in a PCIe 4.0 slot), the model’s tables fetch less (gpt-oss 0,0,0,1,1, Qwen3 0,0,1,1,2,2,3,3,4) and beat the fixed table by 21–34% on gpt-oss and 9–18% on Qwen3, about twice the gain on host B for Qwen3 and four to six times for gpt-oss: the per-machine

Table 17: Can the bound’s read time be reached (job 102)? MIN’s per-layer host reads of gpt-oss (first 4,000 steps of the AIME routing), replayed without the model through CPU threads and the copy engine. Each mode’s share of the bound’s host term (R^*S/B_{host} , ms per token in the column *Term*); *Model*: the per-layer mode predicted from the probe’s rates. Two repetitions of each mode agree within 0.9%.

Host	Link/ CPU	C	Term (ms)	Split per layer	Split per token	Link only	CPU only	Model
Pd again	0.40	14	7.04	86%	93%	35%	92%	98%
Pd again	0.40	32	2.84	89%	93%	35%	92%	98%
9950X	1.02	14	9.70	96%	97%	95%	86%	94%
9950X	1.02	32	3.91	96%	97%	94%	86%	92%
9800X3D	0.55	14	9.56	95%	95%	53%	91%	95%
9800X3D	0.55	32	3.85	93%	94%	53%	91%	94%

Table 18: FETCH tables at 43.75% on Qwen3-30B-A3B (launch 1): entry n is how many of a layer’s n missed experts are copied over PCIe; the rest run on the CPU. Ratio to the fixed table, paired, 95% CI.

Table	$f(0..8)$	tok/s	vs fixed
Lite	0,0,1,1,1,2,2,3,3	109.0	1.057 [1.052, 1.062]
Model	0,0,1,1,2,3,3,4,5	108.0	1.048 [1.041, 1.055]
Model, fetch a single miss	0,1,1,1,2,3,3,4,5	104.1	1.010 [1.007, 1.012]
Fixed	0,1,1,2,3,3,4,5,6	103.1	1
None (CPU only)	0, . . . , 0	100.7	0.977 [0.970, 0.984]
All (fetch every miss)	0,1, . . . , 8	98.7	0.957 [0.953, 0.962]

split matters most where the link is slow. On the Ryzen, ours leads FreeToken by 1.04–1.08 $\times$; with the fixed table it would trail at 12.5%. On the Core i9 the first comparison (job 085) was confounded: FreeToken’s hybrid backend ran gpt-oss at 12–66 tok/s, below its own offload backend (25–86), because its CPU executor spread 23 threads across the 8 performance and 16 efficiency cores and its calibration then sent two thirds of the misses over the slow link, the same failure Appendix C reports for llama.cpp; the 1.4–2.0 $\times$ lead that gave us measured a thread configuration. We reran it (job 088, a second rental of the same CPU model) with FreeToken’s executor on 8 threads pinned to the performance cores: its hybrid backend then runs 1.4–3.1 $\times$ faster than before (37.5, 62.1 and 92.5 tok/s; its calibration now copies 51.7% of misses), beating its own offload backend at 11 and 25% and tying it at 40%, and ours leads its better configuration by 1.58, 1.52 and 1.51 $\times$ (lowest interval bound 1.47). Restricted to the CPU alone, FreeToken’s 8-thread executor decodes 13.7 tok/s at every budget. llama.cpp on the same rental runs 40.0 tok/s with one thread per performance core (−9.4% from the model’s prediction) and 32.8 with all 24 threads (+25.9%): the hybrid-core failure is a thread-count choice, and the model holds once the choice is right. The law predicted llama.cpp on the Ryzen within +2.7%. Figure 5 puts every same-machine comparison on one axis.

Long outputs. Our lead does not come from the first 256 tokens. On gpt-oss, on ten held-out hard problems (AIME 2022, problems 11–15 of both exams) that neither system has finished reasoning about after 2,048 tokens (at most 1 of 10 reach an answer), ours runs 1.36 [1.34, 1.38], 1.32 [1.30, 1.34] and 1.19 [1.17, 1.21] times FreeToken at 11, 25 and 40% over all 2,048 tokens, and 1.37 [1.34, 1.39], 1.33 [1.31, 1.35] and 1.19 [1.17, 1.22] over tokens 257–2,048 (job 083, host A); both engines stream one event per token. On Qwen3, over tokens 257–2,048 of ten held-out MATH-500 problems, ours runs 1.043 [1.020, 1.070], 1.056 [1.036, 1.077] and 1.054 [1.025, 1.086] times FreeToken at 12.5, 25 and

43.75%. Both leads are larger than over the same runs’ first 256 tokens. Turning off FreeToken’s prefill overlap, which lends it cache slots during each prompt, changes its overall speed by 1% or less.

FreeToken’s own headline model. On FreeToken’s own headline model, Qwen3.6-35B-A3B in BF16 (256 experts of 6.3 MB per layer, hybrid linear attention), the two systems are close: ours runs 1.030 [1.023, 1.038] times FreeToken at 12.5% and ties at 25 and 37.5% (1.010 [0.998, 1.021], 1.003 [0.994, 1.013]), at 1.98–2.48× llama.cpp (job 086, host A). FreeToken as shipped, with its automatic backend and cache size, runs this model at 93.8 tok/s on that machine, above the 77–83 tok/s its paper reports on an RTX 5090, so our setup does not handicap it; FreeToken’s fetch cap and thread count were left at their defaults, while our FETCH table was swept. Our lead is therefore model-dependent: largest with few large experts (gpt-oss, top-4 of 128), smallest with many small ones.

FreeToken tuned. On an RTX 5090 with a Ryzen 9 9950X (job 098, one launch, configurations in a shuffled order per cell) FreeToken ran in five settings per cell: hybrid with its calibrated fetch split (its default), hybrid with the fetch cap fixed at one and at two, hybrid with 8 CPU threads instead of one per physical core, and offload. Its best setting was the calibrated hybrid with 8 threads at three cells and offload at the other three (gpt-oss 25 and 40%, Qwen3 43.75%), at most 1.04× its Table 2 setting; the fixed caps were never better than its own split by more than 0.3%. Ours over FreeToken’s best: 1.17, 1.17 and 1.09 on gpt-oss, 1.02, 1.04 and 0.98 on Qwen3, within 0.04 of host S’s ratios at every cell. With every expert on the CPU, stock llama.cpp decodes gpt-oss 1.13× and Qwen3 0.99× as fast as FreeToken’s CPU backend.

Other systems and outputs. Two newer llama.cpp-based systems, Pipelined Sharding [47] and leloch’s expert cache [32], run gpt-oss-120b at 1.28–1.45× stock llama.cpp; ours ran 1.42–2.34× the better of them (job 077). KTransformers [11] (kt-kernel 0.7.0, exact BF16 CPU experts, the same number of experts statically on the GPU) ran Qwen3 in BF16 at 0.98–1.01× stock llama.cpp on an RTX 5090 with a Ryzen 9 9950X, and ours at 2.2–4.4× it (job 078); both static placements read the same CPU expert bytes from the same DRAM. Its published speeds rely on AMX Xeons and INT4/INT8 CPU weights, which that host lacks and which change the model, so this is not its best setting. Teacher-forced on fixed text (1536 steps per model) against stock llama.cpp with every weight in the VRAM of an RTX PRO 6000 (job 090), our cache at 25% has a mean KL divergence over the full vocabulary of 0.0019 nats on gpt-oss and 0.0005 on Qwen3 (99.9th percentile 0.121 and 0.059, maximum 0.21 and 0.13), picks the same next token on 98.5% and 99.3% of steps, and changes the loss by +0.16% and +0.19%. Stock llama.cpp’s own `--n-cpu-moe` placement diverges from the same reference by as much (mean KL 0.0017 and 0.0008, agreement 98.8% and 99.3%): the differences are those of running experts on the CPU (the helpers quantise activations to Q8_0 where the GPU path uses Q8_1), not of the cache. Task accuracy was not measured.

Prompts and batches. The cache serves single-token steps; prompts and batches above one run llama.cpp’s stock graph with every expert in host memory, and pay for the cache’s resident experts and its one-thread CPU side. In llama-batched-bench at 25% on host S (job 089, both models) our prefill at 512 and 2,048 tokens runs 79–83% of stock llama.cpp’s and our decode at 2 and 4 parallel sequences 77–79% of it (the tool feeds both engines the same random tokens).

J The audit’s rows

Table 19 lists the 52 published measurements of Section 3 with their bounds. The sources are Fiddler [30], ProMoE [44], KTransformers [11], HybriMoE [64], Fate [15], SP-MoE [12], MoE-SpeQ [48], the CPU–GPU collaborative inference of Huang et al. [22], FreeToken [55], SeqMoE [50], NVIDIA’s pipelined sharding [47], a llama.cpp fork [32] and a llama.cpp pull request [13]. Three caveats apply to every row: the ceilings are datasheet ones where ours are probed; the trace is the model’s own sampled text, sometimes of a sibling variant (an FP8 trace for a Q4 deployment); and systems whose cache is pooled across layers, whose placement pins whole layers, or which verify several tokens per pass are scored against the per-layer one-token bound, which is not theirs.

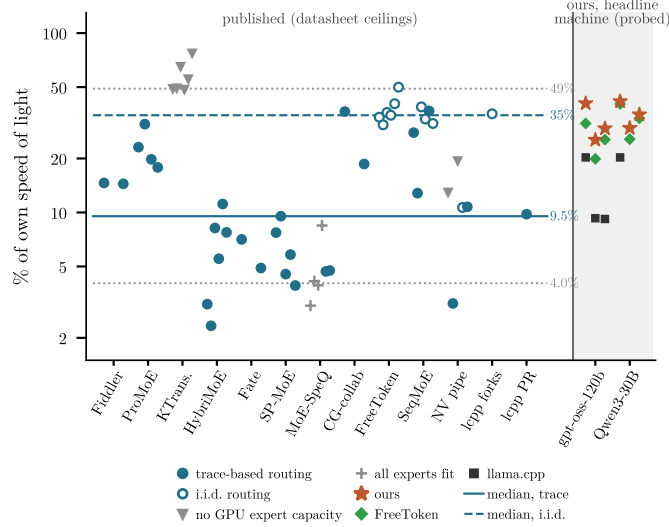


Figure 9: Published MoE-offloading measurements (52 rows from 13 sources) as a percentage of their own bound, computed from the paper’s hardware at datasheet rates and, where a routing trace of the model exists, the per-layer Belady optimum (filled); hollow points assume uniform routing, which overstates the optimum’s reads and so the percentage; the dotted lines are the class medians (no GPU expert capacity, 49.0%; every expert resident, 4.0%). Right: our cache, FreeToken and stock llama.cpp on host B against the probed per-layer limit of Table 2 (the pooled bound FreeToken’s and llama.cpp’s designs allow is 0–10% higher).

Table 19: The audit’s rows (Section 3): every published batch-1 measurement we scored, with the system’s stated hardware, its experts per layer resident on the GPU (C), the model’s experts and top- k , the published speed, the bound at datasheet-physical ceilings (GPU datasheet bandwidth, the top of the host-DRAM band, datasheet PCIe, no latencies; `scripts/audit.py`), how the optimum’s reads were obtained (*trace*: Belady’s MIN with bypass on an on-policy routing trace of the model; *uniform*: independent uniform routing, which overstates the reads and so the percentage; *no slots*: no GPU expert capacity, the CPU-only rate is the bound; *all fit*: every expert resident, nothing offloaded) and the percentage of the bound. Medians: trace 9.5% ($n=29$), uniform 34.8% (11), no slots 49.0% (8), all fit 4.0% (4), all rows 14.5% (52). Rows below 5% are more likely configuration mismatches with the bound’s assumptions (precision, dense layers on the CPU, end-to-end timing) than inefficiency, and are not adjudicated.

System	Model	GPU	C	E/k	tok/s	bound	reads	%
Fiddler [30]	Mixtral-8x7B	Quadro RTX 6000	1	8/2	2.5	17	trace	14.6
	Mixtral-8x7B	RTX 6000 Ada	3	8/2	8.3	57	trace	14.4
ProMoE [44]	DeepSeek-V2-Lite	RTX 4090	45	64/6	49.9	216	trace	23.1
	Qwen1.5-MoE-A2.7B	RTX 4090	46	60/4	67.2	216	trace	31.1
	Qwen2-57B-A14B	RTX 4090	42	64/8	26.9	136	trace	19.8
	Mixtral-8x7B	RTX 4090	6	8/2	27.0	152	trace	17.8
KTransformers [11]	DeepSeek-V3-0324	A100 40GB	0	256/8	5.9	12	no slots	48.8
	DeepSeek-V2.5-1210	A100 40GB	0	160/6	11.9	24	no slots	49.2
	Qwen2-57B-A14B	A100 40GB	0	64/8	22.9	35	no slots	64.9
	DeepSeek-V3-0324	RTX 4080	0	256/8	16.1	33	no slots	48.3
	DeepSeek-V2.5-1210	RTX 4080	0	160/6	17.6	32	no slots	55.3
	Qwen2-57B-A14B	RTX 4080	0	64/8	37.0	48	no slots	76.9
HybriMoE [64]	DeepSeek-V2-Lite-Chat	RTX A6000	16	64/6	14.5	470	trace	3.1
	DeepSeek-V2-Lite-Chat	RTX A6000	48	64/6	15.3	655	trace	2.3
	Mixtral-8x7B-Instruct	RTX A6000	2	8/2	4.4	53	trace	8.2
	Mixtral-8x7B-Instruct	RTX A6000	6	8/2	7.3	133	trace	5.5
	Qwen2-57B-A14B-Instruct	RTX A6000	16	64/8	9.6	86	trace	11.1
	Qwen2-57B-A14B-Instruct	RTX A6000	48	64/8	9.0	116	trace	7.7
Fate [15]	Qwen1.5-MoE-A2.7B	RTX 3090	46	60/4	14.0	198	trace	7.1
	Qwen1.5-MoE-A2.7B	GTX 1080 Ti	15	60/4	3.7	76	trace	4.9
SP-MoE [12]	Mixtral-8x7B	RTX 3090	1	8/2	1.2	16	trace	7.7
	Mixtral-8x7B	RTX 4090	1	8/2	1.3	13	trace	9.5

continued

System	Model	GPU	C	E/k	tok/s	bound	reads	%
MoE-SpeQ [48]	Mixtral-8x7B	A100 40GB	3	8/2	1.5	32	trace	4.5
	Phi-3.5-MoE	RTX 4090	3	16/2	3.3	57	trace	5.8
	DeepSeek-V2-Lite	A100 40GB	7	64/6	5.6	142	trace	3.9
	DeepSeek-V2-Lite	A100 40GB	64	64/6	10.1	335	all fit	3.0
	DeepSeek-V2-Lite	A100 40GB	64	64/6	13.8	335	all fit	4.1
	Qwen1.5-MoE-A2.7B	A100 40GB	60	60/4	13.2	335	all fit	3.9
	Qwen1.5-MoE-A2.7B	A100 40GB	60	60/4	28.3	335	all fit	8.5
	Phi-3.5-MoE	A100 40GB	7	16/2	6.0	129	trace	4.7
CPU-GPU collab. [22]	Phi-3.5-MoE	A100 40GB	7	16/2	6.1	129	trace	4.7
	Mixtral-8x7B	RTX 4090	1	8/2	4.8	13	trace	36.5
	Phi-3.5-MoE	RTX 4090	3	16/2	10.4	56	trace	18.6
FreeToken [55]	Qwen3.6-35B-A3B	RTX 5090	94	256/8	77.1	227	uniform	34.0
	DeepSeek-V4-Flash	RTX 5090	28	256/6	24.9	81	uniform	30.8
	Qwen3.6-35B-A3B	RTX 5090	104	256/8	73.8	205	uniform	36.1
	Qwen3.6-35B-A3B	RTX 4090	72	256/8	42.9	123	uniform	34.8
	Qwen3.6-35B-A3B	RTX 4060 Laptop	60	256/8	39.3	97	uniform	40.4
SeqMoE [50]	GLM-5.2	RTX PRO 6000 Blackwell	36	256/8	14.9	30	uniform	49.9
	Qwen3-30B-A3B-FP8	RTX 4090	57	128/8	104.1	374	trace	27.9
	Qwen3-30B-A3B-FP8	RTX 4090	19	128/8	47.9	374	trace	12.8
	Qwen3.6-35B-A3B-BF16	RTX 5090	102	256/8	118.3	305	uniform	38.8
	Qwen3.6-35B-A3B-BF16	RTX 5090	38	256/8	70.2	212	uniform	33.2
	GPT-OSS-120B-MXFP4	RTX PRO 6000 Blackwell	57	128/4	139.2	378	trace	36.8
	DeepSeek-V4-Flash	RTX PRO 6000 Blackwell	115	256/6	57.4	183	uniform	31.3
	Qwen3-30B-A3B-Instruct-2507	RTX 5090	0	128/8	25.7	199	no slots	12.9
Pipelined sharding [47]	Qwen3-30B-A3B-Instruct-2507	RTX 5090	44	128/8	32.1	1032	trace	3.1
	Qwen3-235B-A22B-Instruct-2507	RTX 5090	0	128/8	7.7	40	no slots	19.3
	Qwen3-235B-A22B-Instruct-2507	RTX 5090	42	128/8	11.5	108	uniform	10.7
	Qwen3-30B-A3B-Instruct-2507	RTX 5070 Ti	107	128/8	54.9	511	trace	10.8
	llama.cpp fork (leloch) [32]	Laguna S 2.1	66	256/10	31.3	88	uniform	35.5
llama.cpp PR 27861 [13]	Qwen3-30B-A3B Q4_K_M derivative	Radeon RX 7600 8GB	32	128/8	16.5	169	trace	9.8

K Artifact description

- **System.** A patch to llama.cpp commit `4da6337` (the expert cache, FETCH, the mailbox and the `llama-ec-bench` instrumentation: teacher-forced loss, logit dumps, routing lookahead records); built with CUDA 13.0 for compute capability 12.0.
- **Jobs.** One shell script per job; each installs FreeToken at commit `0d652e7`, downloads the models from Hugging Face, runs the bandwidth probes, writes any on-machine prediction, runs the client, and stores raw rows (one JSON line per problem), server logs and engine counters. A small runner rents a Vast instance, streams a heartbeat, fetches the results and only then destroys the instance.
- **Analysis.** `scripts/*_stats.py` compute every interval from the raw rows; `scripts/perlayer_model.py`, `speed_limit_v2.py`, `shapley_gap.py`, `policy_study.py`, `foresight_exact.py`, `batchk_trace.py`, `fig_audit_sol.py` and `scorecard.py` produce the model comparison, the limit, the accounting, the trace studies, the audit figure and the scorecard; `scripts/wsg_tables.py` and `wsg_numbers2.py` generate every table, figure and number in this paper from their outputs. The audit’s 52 published rows and the physical speed-of-light model they are scored against are `data/published_measurements.json`, `data/audit/` and `scripts/audit.py`.
- **Cost.** The 88 rentals (53 distinct machines) of jobs 058–101 cost 70.8 US dollars at their hourly GPU prices, by the ledger in the repository; storage and download charges, which the ledger omits, add to this.